

The numodel package*

Paul Zuurbier
mail@paulzuurbier.nl

May 25, 2026

Abstract

A LuaLaTeX package for writing and rendering numerical models (Euler-integrated dynamical systems) directly inside LaTeX documents, aimed at physics teaching material. It provides a text-model pipeline (`\mvar`, `\mrule`, `\computemodel`), Forrester stock-and-flow diagrams, and optional plots of the computed time series via the sibling package `numodel-plot`.

Contents

1	Introduction	1
2	First example: a free-falling ball	2
2.1	<code>\textmodel</code> — rule table	2
2.2	<code>\graphicmodel</code> — Forrester diagram	2
2.3	<code>\computemodel</code> + <code>\diagrammodel</code> — numerical plot	3
3	Configuration	3
3.1	Diagram styles in practice	5
4	Public API	5
4.1	Variables and rules	5
4.2	Expression reference	6
4.3	Render commands	8
4.4	Series accessors	9
4.5	Namespace management	10
4.6	Shared valves	10
5	Variable types	11
6	Generated accessors	11
7	Multiple models in one document	11
8	Requirements	12
9	Implementation	12
9.1	Translation files	55
9.1.1	<code>numodel-EN.def</code> — English (XMILE)	55
9.1.2	<code>numodel-NL.def</code> — Dutch (CoachTaal)	55

1 Introduction

`numodel` lets an author write a dynamical system (stocks, flows, helper variables, rules, and a stop condition) as a sequence of LaTeX macros and renders three complementary views of that model directly in the document:

- a *text model* — a typeset rule table with initial values (`\textmodel`);

*This document corresponds to `numodel` v0.5.0, dated May 25, 2026.

- a *graphic model* — a Forrester stock-and-flow diagram with auto-layout (`\graphicmodel`);
- a *diagram* — a numerical Euler simulation plus PGFPlot of any pair of variables (`\computemodel` followed by `\diagrammodel`; the plot is rendered through the sibling package `numodel-plot`).

All three views are produced from a single set of declarations so the textbook description, the conceptual stock-and-flow diagram, and the numerical result of the same model are guaranteed to stay in sync. Variables and rules live in namespaces (*prefixes*) so a document can contain multiple independent models; `\newmodelprefix{P}` starts a fresh one. The simulation engine runs in Lua (through `luacode`) for $\mathcal{O}(1)$ appends and cheap min/max tracking; the rendering layer is pure `expl3`.

2 First example: a free-falling ball

A ball dropped from $h_0 = 100$ m under constant gravitational acceleration. The complete model:

```
\usepackage[syntax=EN]{numodel}

\newmodelprefix{ball}
\mvar{T}{t}{0}{s}{2}{system}
\mvar{Dt}{dt}{0.1}{s}{2}{system}
\mvar{V}{v}{0}{m\per s}{2}{stock}
\mvar{Y}{y}{100}{m}{3}{stock}
\mvar{G}{g}{-9.81}{m\per s\squared}{3}{aux}

\mrule{V}{\ballV + \ballG * \ballDt}
\mrule{Y}{\ballY + \ballV * \ballDt}
\mrule{T}{\ballT + \ballDt}
\mstop{\ballY <= 0}
```

After the declarations above, three render commands produce the three views shown below, each from the *same* model.

2.1 `\textmodel` — rule table

The verbatim source rendered by `\textmodel`:

	model	initial values
1	$v = v + g \cdot dt$	$t = 0 \text{ s}$
2	$y = y + v \cdot dt$	$dt = 0.10 \text{ s}$
3	$t = t + dt$	$v = 0 \text{ m s}^{-1}$
4	IF $y \leq 0$ THEN STOP ENDIF	$y = 100 \text{ m}$ $g = -9.81 \text{ m s}^{-2}$

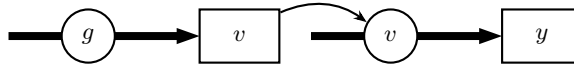
Each `\mvar` with a non-empty start value contributes a row in the *initial values* column; each `\mrule` contributes a row in the *model* column. Symbols come from the second `\mvar` argument (the display text), values are formatted through `siunitx`. `<=` is rendered as $\leq$.

2.2 `\graphicmodel` — Forrester diagram

`\graphicmodel` draws the same model as a stock-and-flow diagram. Stocks (type `stock`) are rectangles; auxiliaries (`aux`) and constants (`constant`) are circles, with two short horizontal dashes flanking the constant ring. In the default `tight` diagram-style, an `aux` or `constant` that is the direct inflow or outflow of a stock is absorbed into the valve label and not drawn as a separate node; `diagram-style=forrester` or `edu` keep both (Section 3).

Flows are inferred per additive term on the right-hand side of each stock rule. After dropping the self-carry term, every `+T` is a candidate inflow and every `-T` a candidate outflow of the target stock; the first non-`system`, non-`Dt` variable appearing inside `T` becomes the flow variable (`aux` and `stock`

beat **constant** when several candidates coexist). When the same flow variable surfaces as an outflow term of stock A and a matching inflow term of stock B, the pair is promoted to a *between-flow*: one valve drains A into B instead of two separate valves. If the flow variable of an inflow term is itself a stock, the source stock takes the valve's place – with a matching outflow term on its own rule (a conserved-quantity between-flow) or, when no such match exists, with a cloud-fed phantom valve plus a causal link to the source stock:



Layout is automatic from the rule graph: stocks (with their inflow/outflow valves) sit on the bottom row, auxiliaries on the middle row, and constants on the top row, each filled left-to-right in declaration order. The order of `\mvar` calls therefore drives the horizontal reading order of the diagram – reordering, inserting or holding back a `\mvar` nudges the visible arrangement without touching any grid coordinates, which is the primary lever a document author has over the look of the diagram. Manual placement remains available through `gridx/gridy` keys on the `\mvar` call (see Section 4). Wide diagrams can be capped to a maximum number of grid columns with `\numodelsetup{gridmaxx=N}`: when a row reaches N, the auto layout shifts the affected items up one row and continues filling.

2.3 `\computemodel` + `\diagrammodel` — numerical plot

`\computemodel` iterates the rules forward in time using Euler integration with step size `\ballDt`, stopping when `\mstop`'s condition becomes true (or when the `maxiter` safety limit is reached, see Section 3). `\diagrammodel{xvar}{yvar}{label}` then plots one variable against another:

```
\computemodel
\diagrammodel{T}{Y}{ball-fall}
```

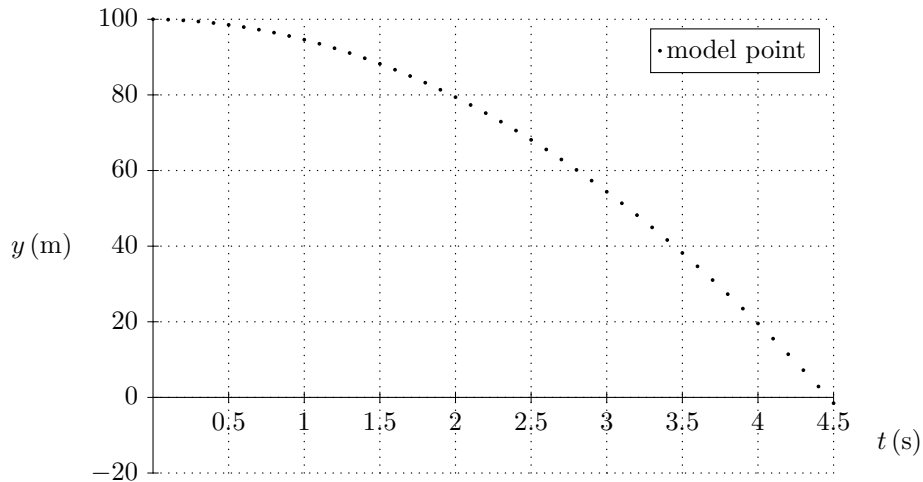


Figure 1: $y(t)$ -diagram

Axis ranges, tick lattice, and labels are computed automatically from the simulated min/max of each variable; the plot inherits the `numodel-plot` style (see that package's documentation for configuration).

3 Configuration

`\numodelsetup` Runtime configuration:

```
\numodelsetup{syntax=NL, maxiter=50000}
```

The same keys can also be passed as package options: `\usepackage[syntax=NL]{numodel}`. Recognised keys:

syntax Language tag for the rule-table rendering. Built-in values:

EN (default) XMI-style ALL-CAPS keywords: IF/THEN/ELSE, AND, OR, ABS, SIGN, ...

NL Dutch CoachTaal keywords: Als/Dan/Anders, EN, OF, Abs, Teken, ...

Each language tag *X* corresponds to a file `numodel-X.def` located via `kpse` when the package processes the key. The package ships with `numodel-EN.def` and `numodel-NL.def`; drop your own `numodel-FR.def` (or any other tag) in `TEXMFHOME/tex/latex/numodel/` and select it with `\usepackage[syntax=FR]{numodel}` – no package rebuild needed. The setting affects display only; the expression syntax in `\mrule` bodies is always `\fp_eval-compatible`.

maxiter Safety limit on the number of `\computemodel` iterations (default 20 000). When reached, the simulation aborts with a warning naming the unmet stop condition.

graphscalex Horizontal grid spacing in centimetres for `\graphicmodel`'s Forrester layout (default 2). Larger values spread the diagram out horizontally.

graphscaley Vertical grid spacing in centimetres for `\graphicmodel`'s Forrester layout (default 2). Larger values spread the diagram out vertically.

`graphscalex/graphscaley` control only the empty space between diagram elements: the nodes themselves (stocks, valves, auxiliaries, constants) have fixed dimensions expressed in `em` and are not individually configurable. Because `em` is relative to the current font size, the way to grow or shrink the nodes uniformly is to change the font size around the `\graphicmodel` call – e.g. wrap it in `{\small \graphicmodel}` or `{\large \graphicmodel}`. Pick spacing with `graphscalex/graphscaley`, pick node size with the surrounding font.

gridmaxx Maximum number of grid columns the auto layout may fill on any one row before wrapping (integer, default 0 = no limit). When the limit is reached, items already placed on the affected row (and everything above it for the stocks row, everything but stocks for the aux row, only the constants for the constants row) shift up by one row to free space, and placement continues from column 0. Manually positioned variables (`\mvar[gridx=...,gridy=...]`) are kept where they are. When wrapping is active the default centring of the aux row and the right-aligning of the stocks row are disabled, so the diagram fills left-to-right, bottom-to-top.

diagram-style Rendering style for the case where a helper or constant is the direct inflow/outflow of a stock. Three values:

tight (default) the valve takes the helper's/constant's label; the helper/constant itself is not drawn as a separate node. Compact and most LaTeX-native.

forrester Forrester/Sterman convention: the valve is drawn without a label and the helper/constant remains as a separate node connected to the valve by a causal arrow.

edu Didactic dual form: the valve carries the label *and* the helper/constant is drawn as a separate node with a causal arrow to the valve. Visually busy but pedagogically explicit.

flowarrow-style Visual style of the flow pipe. **hollow** renders the classic Forrester double-line pipe with an open arrow head; **filled** renders a thick solid arrow. The default tracks **diagram-style**: **forrester** picks **hollow**, the other styles pick **filled**. An explicit value overrides this coupling.

valve-style Visual style of the valve node. **valve** draws the bow-tie/butterfly icon (Forrester); **circle** draws an empty circle on the flow pipe; **edu** draws a labelled circle (the flow variable's display text inside). The default tracks **diagram-style**: **forrester** picks **valve**, the other styles pick **edu**.

flowarrow-cloud-tip Whether the open end of an inflow or outflow pipe is anchored to a cloud node, signalling the model boundary. Default tracks **diagram-style**: **forrester** picks **true**, the other styles pick **false**. May be set globally via `\numodelsetup`, per-render via `\graphicmodel`, or per-stock via `\mvar[flowarrow-cloud-tip=...]`. The most specific source wins.

units Whether the *initial values* cells in `\textmodel` display the SI unit alongside the value (`\qty`) or only the numeric value (`\num`). Boolean, default **true**. May also be supplied to `\textmodel[units=false]` as a per-table override; the global setting is restored after rendering.

tblrenv Which `tabularray` environment wraps the `\textmodel` table. Three values: `longtblr` (default, page-breakable); `tblr` (inline, no page breaks); `talltblr` (inline, no page breaks, supports `\caption/notes`). Pick `tblr` or `talltblr` when `\textmodel` sits inside an enclosing environment that suppresses page breaks (`subfigure`, `minipage`, ...); the default `longtblr` would otherwise emit spurious “(Continued)” / *Continued on next page* markers in that setting. May also be supplied per render as `\textmodel[tblrenv=...]`; the global setting is restored afterwards.

decimal-separator Decimal mark used by every number that `numodel` renders: the *initial values* column of `\textmodel`, and the tick labels of `\diagrammodel`. Two values:

comma use a comma (siunitx output-decimal-marker={,}, pgfplots/pgf/number format/use comma).

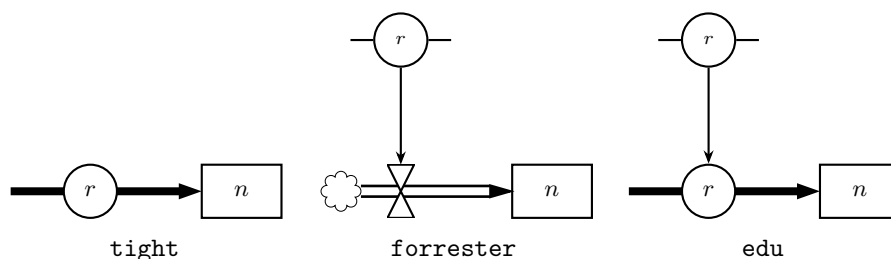
point use a full stop (siunitx output-decimal-marker={.}, pgfplots/pgf/number format/use period).

The default tracks `syntax`: NL picks `comma`, EN picks `point`. Other language files can publish a default by defining `\__numodel_kw_<LANG>_dsep_default`: (expanding to `point` or `comma`); when the macro is absent the default is `point`. An explicit `decimal-separator` key locks that choice and overrides any future `syntax` change. The override is scoped: `numodel` applies it only inside its own renderers (siunitx’s state is restored on group exit), so a document-wide `\sisetup` is not perturbed.

3.1 Diagram styles in practice

The same model rendered under each of the three `diagram-style` values. The model is the simplest case that distinguishes the styles: one stock N with a constant inflow R :

```
\newmodelprefix{flux}
\mvar{T}{t}{0}{\s}{2}{system}
\mvar{Dt}{dt}{1}{\s}{2}{system}
\mvar{N}{n}{0}{\s}{0}{stock}
\mvar{R}{r}{5}{\per\s}{2}{constant}
\mrule{N}{\fluxN + \fluxR * \fluxDt}
\mrule{T}{\fluxT + \fluxDt}
\mstop{\fluxT >= 5}
\graphicmodel[diagram-style=tight]
\graphicmodel[diagram-style=forrester]
\graphicmodel[diagram-style=edu]
```



- **tight** collapses the constant R into the valve label, producing the most compact diagram.
- **forrester** keeps the canonical System-Dynamics convention: unlabelled bow-tie valve, the constant remains a separate node, the link from R to the valve is a thin causal arrow.
- **edu** is a didactic dual: the valve carries the label *and* and the constant remains as a separate node with a causal arrow. Less compact but pedagogically explicit – useful when first introducing the stock/flow vocabulary.

4 Public API

4.1 Variables and rules

`\mvar` Declares a model variable. Signature:

```
\mvar[<keys>]{<Name>}{<text>}{<start>}{<unit>}{<sig>}{<type>}
```

where $\langle Name \rangle$ is a short alphabetic identifier (the prefix-qualified accessor becomes $\backslash\langle prefix \rangle\langle Name \rangle$, along with a family of suffixed accessor macros $\backslash\langle prefix \rangle\langle Name \rangle\langle suffix \rangle$ documented in Section 6), $\langle text \rangle$ is the math-mode display symbol used in the rule table and diagram (e.g. F_{res}), $\langle start \rangle$ is the initial value (a number, or empty for helpers computed by a rule, or any `expl3` fp-evaluable expression involving previously defined model variables), $\langle unit \rangle$ is a bare `siunitx` unit macro sequence (e.g. $\backslash m\backslash per\backslash s\backslash squared$), $\langle sig \rangle$ is the number of significant figures used by $\backslash\langle prefix \rangle\langle Name \rangle num/qty$, and $\langle type \rangle$ is one of `stock`, `aux`, `constant`, or `system`. Each English type also accepts a Dutch alias (`voorraad`, `hulp`, `constante`, `systeem`) for backwards compatibility with existing teaching material. See Section 5.

Naming caveat. All accessor macros of a given prefix share one flat TeX namespace: `\mvar{X}` generates $\backslash\langle prefix \rangle X$ and every $\backslash\langle prefix \rangle X\langle suffix \rangle$ listed in Section 6. Pick $\langle Name \rangle$ s so that no name equals another name followed by such a suffix, otherwise the two definitions silently overwrite each other. The canonical trap is declaring both `T` and `Tmax`: the `max`-suffix accessor of `T` (the extremum $\backslash\langle prefix \rangle Tmax$) collides with the no-suffix accessor of `Tmax`, so $\backslash\langle prefix \rangle Tmax$ inside a rule expression ends up meaning whichever was declared last. The same hazard applies to prefix choices – avoid prefixes where one is a concatenation of another with a variable name (e.g. `ball` with a variable `Y` versus a prefix `ballY`).

Optional $\langle keys \rangle$:

prefix Override the current prefix for this single call.

gridx, gridy Manual placement in the `\graphicmodel` grid; integers, `-1` leaves the slot to auto-layout (default).

alias Math-mode token list that replaces the entire *initial values* cell.

aliasleft, aliasright Replace just the left symbol or right value half of the cell.

\mrule Adds a rule of the form $\langle LHS \rangle \leftarrow \langle expr \rangle$. Signature:

`\mrule*[\langle keys \rangle]{\langle LHS \rangle}{\langle expr \rangle}`

Both forms add the rule to *both* the rule table (`\textmodel`) and the simulation (`\computemodel`); execution is identical. The star only changes the typeset layout when $\langle expr \rangle$ is a ternary `cond ? a : b`. Without the star the ternary is rendered on a single table row,

IF `cond` THEN `lhs = a` ELSE `lhs = b` ENDIF

which is compact but wide. With the star (`\mrule*`) the same ternary is broken across rows,

IF `cond` THEN
`lhs = a`
 ELSE
`lhs = b`
 ENDIF

keeping the table column narrow so a `\graphicmodel` can sit alongside it, and making the source itself easier to read. For non-ternary expressions the star has no effect. $\langle expr \rangle$ may use the full `\fp_eval` expression grammar. See Table 2 (Section 4.2) for a complete overview of supported operators and functions with their XMILE and CoachTaal equivalents.

\mruletext Inserts a free-text row in the rule table without registering a rule with the simulator. Signature: `\mruletext[\langle keys \rangle]{\langle text \rangle}`. Useful for inserting comments or section dividers in long rule tables.

\mstop Sets the simulation stop condition. Signature: `\mstop[\langle keys \rangle]{\langle expr \rangle}`. The simulation halts at the first step where $\langle expr \rangle$ evaluates true. Exactly one `\mstop` per model prefix is required before `\computemodel`. Without one, `\computemodel` issues a warning.

4.2 Expression reference

Table 2 lists all expression constructs for `\mrule` bodies. The **XMILE** column shows XMILE v1.0 syntax, **l3fp** shows the `\fp_eval`-compatible form used inside `\mrule`, and **CoachTaal** shows the Coach7 equivalent (function names from the standard CoachTaal math reference; note that argument separators in CoachTaal are semicolons). **Orange** entries are not yet fully supported by `numodel` (the expression computes correctly, but the rendered keyword in the rule table is not yet translated). *Italic* cells contain a derived equivalent rather than a native keyword.

Table 2: Expression reference: XMILE, \fp_eval, and CoachTaal

XMILE	l3fp (\fp_eval)	CoachTaal
<i>Arithmetic operators</i>		
+	+	+
-	-	-
*	*	*
/	/	/
^	^	^
MOD(x,y)	$x - \text{trunc}(x/y)*y$	$x - \text{Entier}(x/y)*y$
<i>Comparison operators</i>		
<	<	<
<=	<=	<=
>	>	>
>=	>=	>=
=	=	=
<>	!=	<>
<i>Boolean operators</i>		
AND	&&	EN
OR		OF
NOT	!	NIET
<i>Control flow</i>		
IF c THEN a ELSE b	c ? a : b	Als c Dan a Anders b EindAls
<i>General math functions</i>		
ABS(x)	abs(x)	Abs(x)
SIGN(x)	sign(x)	Teken(x)
SQRT(x)	sqrt(x)	Sqrt(x)
INT(x)	trunc(x)	$\text{Entier}(\text{Abs}(x))*\text{Teken}(x)$
INT(x+0.5)	round(x)	Round(x)
INT(x)	floor(x)	Entier(x)
-INT(-x)	ceil(x)	$-\text{Entier}(-x)$
MIN(x,y)	min(x,y)	Min(x;y)
MAX(x,y)	max(x,y)	Max(x;y)
—	fact(x)	Fac(x)
INT(LOG10(ABS(x)))	logb(x)	$\text{Entier}(\text{Log}(\text{Abs}(x)))$
<i>Exponential and logarithmic</i>		
EXP(x)	exp(x)	Exp(x)
LN(x)	ln(x)	Ln(x)
LOG10(x)	$\ln(x)/\ln(10)$	Log(x)
<i>Trigonometry (radians)</i>		
SIN(x)	sin(x)	Sin(x)
COS(x)	cos(x)	Cos(x)
TAN(x)	tan(x)	Tan(x)
ARCSIN(x)	asin(x)	Arcsin(x)
ARCCOS(x)	acos(x)	Arccos(x)
ARCTAN(x)	atan(x)	Arctan(x)
ARCTAN2(y,x)	atan(y,x)	$\text{Arctan}(y/x)$
1/TAN(x)	cot(x)	$1/\text{Tan}(x)$
1/SIN(x)	csc(x)	$1/\text{Sin}(x)$
1/COS(x)	sec(x)	$1/\text{Cos}(x)$
ARCSIN(1/x)	acsc(x)	$\text{Arcsin}(1/x)$

Continued on next page

Table 2: Expression reference: XMILE, \fp_eval, and CoachTaal (Continued)

XMILE	l3fp (\fp_eval)	CoachTaal
<i>ARCCOS(1/x)</i>	<i>asec(x)</i>	<i>Arccos(1/x)</i>
<i>ARCTAN(1/x)</i>	<i>acot(x)</i>	<i>Arctan(1/x)</i>
<i>ARCTAN2(x,y)</i>	<i>acot(y,x)</i>	<i>Arctan(x/y)</i>
<i>Trigonometry (degrees)</i>		
<i>SIN(Pi/180*x)</i>	<i>sind(x)</i>	<i>Sin(Pi/180*x)</i>
<i>COS(Pi/180*x)</i>	<i>cosd(x)</i>	<i>Cos(Pi/180*x)</i>
<i>TAN(Pi/180*x)</i>	<i>tand(x)</i>	<i>Tan(Pi/180*x)</i>
<i>180/Pi*ARCSIN(x)</i>	<i>asind(x)</i>	<i>180/Pi*Arcsin(x)</i>
<i>180/Pi*ARCCOS(x)</i>	<i>acosd(x)</i>	<i>180/Pi*Arccos(x)</i>
<i>180/Pi*ARCTAN(x)</i>	<i>atand(x)</i>	<i>180/Pi*Arctan(x)</i>
<i>180/Pi*ARCTAN2(y,x)</i>	<i>atand(y,x)</i>	<i>180/Pi*Arctan(y/x)</i>
<i>1/TAN(Pi/180*x)</i>	<i>cotd(x)</i>	<i>1/Tan(Pi/180*x)</i>
<i>1/SIN(Pi/180*x)</i>	<i>cscd(x)</i>	<i>1/Sin(Pi/180*x)</i>
<i>1/COS(Pi/180*x)</i>	<i>secd(x)</i>	<i>1/Cos(Pi/180*x)</i>
<i>180/Pi*ARCSIN(1/x)</i>	<i>acscd(x)</i>	<i>180/Pi*Arcsin(1/x)</i>
<i>180/Pi*ARCCOS(1/x)</i>	<i>asecd(x)</i>	<i>180/Pi*Arccos(1/x)</i>
<i>180/Pi*ARCTAN(1/x)</i>	<i>acotd(x)</i>	<i>180/Pi*Arctan(1/x)</i>
<i>180/Pi*ARCTAN2(x,y)</i>	<i>acotd(y,x)</i>	<i>180/Pi*Arctan(x/y)</i>
<i>Constants</i>		
PI	pi	Pi
e	<i>exp(1)</i>	<i>Exp(1)</i>
INF	inf	—
NAN	nan	—
<i>PI/180</i>	deg	<i>Pi/180</i>
<i>1</i>	true	Aan
<i>0</i>	false	Uit
<i>Simulation-specific functions (XMILE only)</i>		
TIME	<i>user variable (\mvar)</i>	<i>user variable</i>
DT	<i>user variable (\mvar)</i>	<i>user variable</i>
STEP(h,t₀)	<i>T >= t₀ ? h : 0</i>	<i>Als T >= t₀ Dan h Anders 0 EindAls</i>
RAMP(s,t₀)	<i>T > t₀ ? s*(T-t₀) : 0</i>	<i>Als T > t₀ Dan s*(T-t₀) Anders 0 EindAls</i>
DELAY(x,dt)	<i>not supported</i>	<i>not supported</i>
SMOOTH(x,t)	<i>not supported</i>	<i>not supported</i>
INIT(x)	<i>not supported</i>	<i>not supported</i>
PREVIOUS(x)	<i>not supported</i>	<i>not supported</i>
<i>RANDOM(0,1)</i>	rand()	Rand
<i>RANDOM(lo,hi)</i>	<i>lo + (hi-lo)*rand()</i>	<i>lo + (hi-lo)*Rand</i>
<i>not supported</i>	randint(n)	<i>not supported</i>
<i>not supported</i>	randint(m,n)	<i>not supported</i>

4.3 Render commands

`\textmodel` Renders the rule-and-startvalue table. Optional [`<keys>`] accepts `prefix=<name>` (render a non-current model), `units=true` or `units=false`, and `tblrenv=tblrlongtblrtalltblr`. Each per-call key overrides the global `\numodelsetup` setting for this single render only; the global state is restored afterwards.

`tblrenv` selects which `tabularray` environment wraps the table: `longtblr` (default) breaks across pages, `tblr` renders inline without page breaks, `talltblr` renders inline but with caption and note support. Use `tblr` or `talltblr` when `\textmodel` sits inside an enclosing environment that suppresses page breaks (`subfigure`, `minipage`, ...); the default `longtblr` would otherwise emit spurious continuation markers there.

Row spacing. `numodel` loads `tabularray` and sets `\SetTblrInner{rowsep=0pt}` globally so the rule listing renders compactly. This applies to *every* `tabularray` table in the document; if you want the default spacing back in your own tables, issue `\SetTblrInner{rowsep=2pt}` (or whatever value you prefer) somewhere in your document.

`\graphicmodel` Renders the Forrester stock-and-flow diagram. Variables of type `stock` become rectangles, `constant` become circles, `aux` become identifier nodes; flow arrows connect stocks to constant or helper sources/sinks based on which variables appear in the right-hand side of stock-updating rules.

Optional `[<keys>]` accepts `prefix=<name>` (render a non-current model) and `diagram-style=tightforrestered` which overrides the global `\numodelsetup` setting for this single render only. The global state is restored afterwards, so multiple `\graphicmodel` calls can each pick their own style without re-issuing `\numodelsetup`.

`\computemodel` Runs the Euler simulation. Each iteration step is executed in LaTeX itself: expl3's `\fp_eval` (the engine behind the LaTeX2e `\fpeval`) evaluates the stop condition and then every `\mrule` body in declaration order, writing each new value into the accessor macro `\<prefix><Name>`. The per-step values are appended to Lua tables on the side – storing the time series as growing TeX token lists would cost $O(N)$ per append and $O(N^2)$ overall, whereas the Lua-table append is $O(1)$, so a 20 000-step run stays linear in total work. Lua also keeps the running min and max so no second pass over the series is needed. After `\computemodel` returns, `\<prefix><Name>min` / `\<prefix><Name>max` hold the extrema, `\<prefix><Name>` holds the final-step value, and the full time-series can be retrieved with `\mcoords` / `\mstep`.

`\diagrammodel` Convenience wrapper that produces a complete figure with caption and label. Signature:

`\diagrammodel[<keys>]{<xvar>}{<yvar>{,...}}[<extra>]{<label>}`

Reads min/max and display text from `\<prefix><xvar>` etc., delegates to `\drawplot` from `numodel-plot`, and emits `\caption{$y(x)$-diagram}\label{fig:<label>}`. The optional `<extra>` argument is appended to the `axis` body, useful for additional `\addplot` lines (annotations, theoretical curves). Must be called after `\computemodel`.

The `<yvar>` argument accepts a comma-separated list of variables: every entry whose `unitraw` matches the first one's is drawn into the same diagram as discrete model points (no connecting lines), with the y-axis range scaled to the joint min/max of the kept series. Entries with a non-matching unit are dropped and a warning is issued. The seven colours cycle through Okabe & Ito's colour-blind safe palette (yellow omitted), ordered so consecutive series differ in luminance – which keeps them distinguishable on a greyscale printout as well. The legend lists each kept variable's display text.

4.4 Series accessors

`\mcoords` Returns a comma-separated PGFPlots coordinate list of the simulated series for two variables. Fully expandable. Signature: `\mcoords{<xvar>}{<yvar>}` (current prefix); `\mcoordsp{<prefix>}{<xvar>}{<yvar>}` (explicit prefix). Both forms are usable inside `\addplot coordinates{ ... }`.

`\mstep` Returns the value of one variable at a chosen iteration. Fully expandable. Signature: `\mstep{<Name>}{<i>}`; `\mstepp{<prefix>}{<Name>}{<i>}`. The current prefix is prepended automatically by `\mstep`. Step indexing is 0-based: step 0 is the initial-values row, step $N-1$ is the final recorded step after `\computemodel`. Negative indices count from the end (Python-style), so `\mstep{Y}{-1}` is the last recorded y and `\mstep{Y}{-2}` is the penultimate one. Returns nothing (silently) if the index is out of range.

Example – a red secant line through the last two simulated (t, y) points of the free-fall ball (the model from the first example), extrapolated across the whole t -domain and labelled in the legend. The slope is the rise-over-run of the last two steps, the intercept is the final y -value:

```
\diagrammodel{T}{Y}{%
  \addplot[red, very thick, domain=\ballTmin:\ballTmax]
    { \mstep{Y}{-1}
      + ( \mstep{Y}{-1} - \mstep{Y}{-2} )
      / ( \mstep{T}{-1} - \mstep{T}{-2} )
      * ( x - \mstep{T}{-1} ) };
  \addlegendentry{secant endpoints}
}{ball-fall-secant}
```

Inside the optional [`<extra>`] argument `\mstep` expands as a literal number into PGFPlots' math parser, so each `\mstep` is evaluated only once (when the expression is constructed) rather than per sample. Applied to the `ball` model:

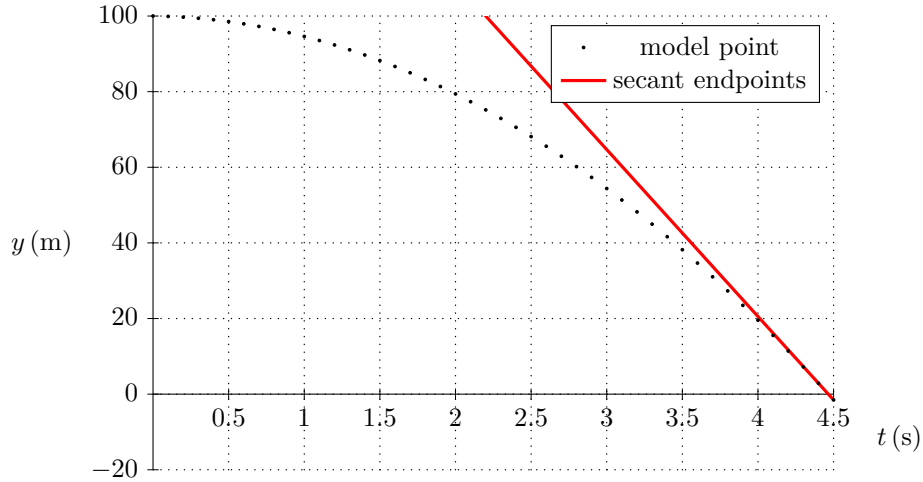


Figure 2: $y(t)$ -diagram

4.5 Namespace management

- `\newmodelprefix` Creates a new model namespace and switches to it. Signature: `\newmodelprefix{<name>}`. Subsequent `\mvar` / `\mrule` / `\mstop` calls bind to this prefix. All generated accessors are prefixed (so `\mvar{Y}{y}{...}{...}{...}{...}` under prefix `ball` produces `\ballY`, `\ballYtext`, etc.).
- `\switchmodelprefix` Switches to a previously created prefix. Useful when a document defines several models early and renders them later out of order. Signature: `\switchmodelprefix{<name>}`.
- `\NumodelForEachVar` Iterates a token list over every registered variable across every known prefix. Signature: `\NumodelForEachVar{<code with #1>}`; inside the body, `#1` is the full prefixed name (e.g. `ballY`). Used by external tools (e.g. a worksheet system) that need to expand every model accessor.

4.6 Shared valves

When the same auxiliary variable appears as the inflow of more than one stock, `\graphicmodel` draws a single valve next to the first such stock and threads a curved branch (`to[bend left=30]` – the same bend as the curved causal arrows) from that valve over the primary stock into each additional one. The auto-layout leaves the involved stocks side by side on the same row so the branch stays compact. A minimal example:

```
\mvar{R}{r}{5}{\m\cubed\per\s}{2}{aux}
\mvar{Va}{V_1}{0}{\m\cubed}{3}{stock}
\mvar{Vb}{V_2}{0}{\m\cubed}{3}{stock}
\mrule{Va}{\Va + \R * \Dt}
\mrule{Vb}{\Vb + \R * \Dt}
```

renders one `R` valve, a straight pipe $R \rightarrow V_a$, and a curved branch $R \rightarrow V_b$ arcing over V_a . (Note: TeX control words do not accept digits, so the macro names `Va/Vb` are used internally while the display texts `V_1/V_2` appear in the rendered diagram and table.)

Shared *outflows* are handled symmetrically. When one variable drains more than one stock, the valve is placed to the right of the last-declared source stock, and every earlier source attaches with a curved branch arcing over the intervening stocks into the shared valve. This matters in particular for physical models where a single Stefan–Boltzmann constant or friction coefficient drains several reservoirs into the same sink.

Two flow-classification refinements support these layouts:

- The flow-variable heuristic prefers **aux** and **stock** variables over **constants** when both appear in the same additive term, so an inflow valve carries the meaningful rate variable instead of an earlier-declared scaling constant.
- Top-level parenthesised inflow terms of the form $(A - B)*C$ are distributed into $+A*C$, $-B*C$ before classification, so **A** surfaces as the inflow valve and **B** (typically a radiative or dissipative loss involving a constant) surfaces as the outflow valve.

5 Variable types

The sixth `\mvar` argument (*<type>*) tags a variable with a role. The type drives both `\textmodel` layout (whether the row appears in *initial values*) and `\graphicmodel` node shape. Each type has a canonical English name and a Dutch alias; the two are interchangeable.

stock (alias *voorraad*) A stock that accumulates over time. Drawn as a rectangle in the Forrester diagram; its rule must be of the form $\text{stock} \leftarrow \text{stock} + \dots$ so that the integrator can detect inflows and outflows. Receives an *initial values* row.

constant (alias *constante*) A constant parameter (mass, gravitational acceleration, spring stiffness, ...). Drawn as a circle. Receives an *initial values* row.

aux (alias *hulp*) An auxiliary variable computed from other variables on each step. Drawn as a plain identifier node, no rectangle. No *initial values* row (start value is normally left empty).

system (alias *systeem*) System-level bookkeeping (time **T**, step size **Dt**, terminal time, ...). Drawn separately, no flow arrows. Receives an *initial values* row.

6 Generated accessors

Each `\mvar` call generates a family of accessor macros named `\<prefix><Name><suffix>`. The full set:

(no suffix) Current numeric value (post-rule-update if inside a step, post-final-step after `\computemodel`).

text The display symbol passed as the second argument of `\mvar`.

unit `\unit{...}` applied to the unit argument (siunitx-formatted).

unitraw The unit argument verbatim, without the `\unit{}` wrapper, suitable for use as a building block (e.g. `\xlabelunit` in `numodel-plot`).

num The current value formatted via siunitx's `\num{}` with the variable's significant figures. Used by `\textmodel` in the *initial values* column when `units=false`.

qty As **num**, but including the unit (`\qty{value}{unit}`). Used by `\textmodel` in the *initial values* column when `units=true` (the default).

pre As **qty**, but with engineering prefix mode (e.g. `1500 W` renders as `1{,}5\,kW`).

sign Significant-figure count (raw integer).

type Variable type (raw string).

min, **max** Extrema over the simulated series. Empty before `\computemodel`.

gridx, **gridy** Manual placement coordinates in the Forrester grid; `-1` if left to auto-layout.

alias, **aliasleft**, **aliasright** Override tokens for the *startwaarden* cell layout.

7 Multiple models in one document

Each `\newmodelprefix` starts a fresh namespace. This lets a document contain several unrelated models without name clashes:

```
\newmodelprefix{ball}
\mvar{Y}{y}{100}{\m}{3}{stock}
% ... ball model ...

\newmodelprefix{spring}
\mvar{X}{x}{0.1}{\m}{3}{stock}
% ... spring model ...

% Render the ball model:
\switchmodelprefix{ball}\textmodel\computemodel\diagrammodel{T}{Y}{fall}

% Render the spring model:
\switchmodelprefix{spring}\textmodel\computemodel\diagrammodel{T}{X}{spring}
```

Each prefix carries an independent rule list, stop condition, and recorded series.

8 Requirements

numodel requires LuaLaTeX (the engine, for the Lua runtime) and TeX Live 2022 or later. Mandatory dependencies: `expl3`, `xparse`, `l3keys2e`, `amsmath`, `amssymb`, `tikz`, `luacode`, `siunitx`, `float`, and the sibling package `numodel-plot` (which itself pulls in `pgfplots`). The companion Lua module `numodel.lua` must be installed alongside the `.sty` in a directory searched by `kpse`.

9 Implementation

```
1 (*package)
2 \NeedsTeXFormat{LaTeX2e}
3 \ProvidesPackage{numodel}[2026/05/23 v0.5.0]
4 Numerical physics models with Euler integration and Forrester diagrams]
5
6 \RequirePackage{expl3}
7 \RequirePackage{xparse}
8 \RequirePackage{l3keys2e}
9 \RequirePackage{amsmath}
10 \RequirePackage{amssymb} % \leqslant / \geqslant in rendered <= / >=
11 \RequirePackage{tikz}
12 \usetikzlibrary{shapes.symbols}
13 \usetikzlibrary{arrows.meta}
14 % Custom Cloud arrow tip used as the open end of inflow / outflow
15 % pipes. The silhouette consists of nine concatenated elliptical
16 % arcs (\pgfpatharc): each arc starts at the current path point,
17 % which pgf treats as lying on an ellipse at angle <start>, and
18 % sweeps through to angle <end>. Varying the start and end angles
19 % per arc produces zigzagging half-circle puffs -- the
20 % characteristic cloud outline. A single factor \puffscale sets the
21 % rx and ry of every ellipse and thus the size of all puffs at once.
22 % The chain has a horizontal span of (4+2*sqrt(2))*rx ~ 6.83*rx;
23 % with puffscale=0.22 that is ~ 1.5*\pgfarrowlength. The path
24 % therefore starts at -0.5*\pgfarrowlength so the silhouette covers
25 % [-0.5L, L] in local arrow coordinates and the apex lands exactly
26 % at \pgfarrowlength -- where the built-in tips (Latex, Stealth,
27 % ...) also place their tip. The line width is set explicitly to
28 % 0.3pt, independent of the arrow's line width, so a thick arrow
29 % does not also get a thick cloud outline.
30 \pgfdeclarearrow{
31   name = Cloud,
32   parameters = { \the\pgfarrowlength \the\pgfarrowwidth },
33   setup code = {
34     \pgfarrowssettipend{\pgfarrowlength}
```

```

35 \pgfarrowssetbackend{-0.5\pgfarrowlength}
36 \pgfarrowssetlineend{-0.5\pgfarrowlength}
37 \pgfarrowssetvisualbackend{-0.5\pgfarrowlength}
38 \pgfarrowssetvisualtipend{\pgfarrowlength}
39 },
40 defaults = { length = 1em, width = 1em },
41 drawing code = {
42 \def\puffscale{0.22}
43 \edef\puffrx{\puffscale\pgfarrowlength}
44 \edef\puffry{\puffscale\pgfarrowwidth}
45 \pgfsetlinewidth{0.3pt}
46 \pgfpathmoveto{\pgfqpoint{-0.5\pgfarrowlength}{0pt}}
47 \pgfpatharc{180}{ 90}{\puffrx\space and \puffry}
48 \pgfpatharc{225}{ 45}{\puffrx\space and \puffry}
49 \pgfpatharc{180}{ 0}{\puffrx\space and \puffry}
50 \pgfpatharc{135}{-45}{\puffrx\space and \puffry}
51 \pgfpatharc{ 90}{-90}{\puffrx\space and \puffry}
52 \pgfpatharc{ 45}{-135}{\puffrx\space and \puffry}
53 \pgfpatharc{ 0}{-180}{\puffrx\space and \puffry}
54 \pgfpatharc{-45}{-225}{\puffrx\space and \puffry}
55 \pgfpatharc{-90}{-180}{\puffrx\space and \puffry}
56 \pgfpathclose
57 \pgfusepathqfillstroke
58 },
59 }
60 \RequirePackage{luacode}
61 \RequirePackage{siunitx}
62 \RequirePackage{float} % \diagrammodel uses [H] placement
63 \RequirePackage{tabularray} % \textmodel uses longtblr (page-breakable,
64 % works inside floats/subfigure)
65 % Define a numodel theme for longtblr in \textmodel. The default
66 % 'normal' middlehead/lasthead prefixes the continuation marker with
67 % 'Table N:' from \tablename/\thetable; we don't issue \caption, so we
68 % want only the conthead text (which is itself localised via
69 % \tblrcontheadname, updated by \_numodel_refresh_kw:). Likewise for
70 % the foot. Activated via 'theme = {numodel}' in the longtblr options.
71 \DefTblrTemplate {firsthead} {numodel} {}
72 \DefTblrTemplate {middlehead, lasthead} {numodel} { \UseTblrTemplate {conthead} {default} }
73 \DefTblrTemplate {firstfoot, middlefoot} {numodel} { \UseTblrTemplate {contfoot} {default} }
74 \DefTblrTemplate {lastfoot} {numodel} {}
75 \NewTblrTheme {numodel}
76 {
77 \SetTblrTemplate {firsthead} {numodel}
78 \SetTblrTemplate {middlehead, lasthead} {numodel}
79 \SetTblrTemplate {firstfoot, middlefoot} {numodel}
80 \SetTblrTemplate {lastfoot} {numodel}
81 }
82 % Tighten the default row separation of all tabularray tables: the
83 % \textmodel rule listing is denser and more compact than the
84 % tabularray default. Users who want the default spacing back in
85 % their own tables can simply issue \SetTblrInner{rowsep=<value>} in
86 % their document.
87 \SetTblrInner{rowsep=0pt}
88 \RequirePackage{numodel-plot}
89
90 % =====
91 % Non-expl3 helpers: '\nm@def@num@cs' / '\nm@def@qty@cs' / '\nm@def@pre@cs'
92 % define '\<fullname><suffix>' macros whose body contains literal
93 % siunitx options. Defined OUTSIDE \ExplSyntaxOn so the option-string
94 % colons (~-3:n~, ~-0:0~) and other punctuation keep normal catcodes.
95 % Storing those bodies inside an expl3-context would tokenise '~' as
96 % letter, which then breaks siunitx's option parser at expansion time.
97 % =====
98 \newcommand{\NumodelDefNumCs}[3]{% #1 = fullname, #2 = start expr, #3 = sigfigs
99 \expandafter\gdef\csname #1num\endcsname{%
100 \num[evaluate-expression=true,

```

```

101         round-mode=figures, round-precision=#3,
102         exponent-mode=threshold, exponent-thresholds=-3:#3]
103     {#2}\relax
104 }%
105 }
106 \newcommand{\NumodelDefQtyCs}[4]{% #1 = fullname, #2 = expr, #3 = sigfigs, #4 = unit
107     \expandafter\gdef\csname #1qty\endcsname{%
108         \qty[evaluate-expression=true,
109             round-mode=figures, round-precision=#3,
110             exponent-mode=threshold, exponent-thresholds=-3:#3]
111         {#2}{#4}\relax
112     }%
113 }
114 \newcommand{\NumodelDefPreCs}[4]{% same parameters as above
115     \expandafter\gdef\csname #1pre\endcsname{%
116         \qty[evaluate-expression=true,
117             round-mode=figures, round-precision=#3,
118             prefix-mode=combine-exponent,
119             exponent-mode=engineering,
120             exponent-thresholds=-0:0]
121         {#2}{#4}\relax
122     }%
123 }
124
125 \ExplSyntaxOn
126
127 % =====
128 % Lua module for data storage (O(1) append, O(N) total)
129 % =====
130 % All variable values are stored per step in Lua tables. This
131 % replaces the O(N^2) \xdef accumulation for coordinates and makes
132 % min/max tracking efficient. After \computemodel the results are
133 % pushed back to TeX macros.
134
135 \begin{luacode*}
136 local f = kpse.find_file("numodel.lua", "tex")
137 if f and f ~= "" then
138     dofile(f)
139 else
140     tex.error("numodel: cannot find companion Lua module numodel.lua."
141         .. " Install it in a directory searched by kpse"
142         .. " (e.g. TEXMFHOME/tex/lualatex/numodel/).")
143 end
144 \end{luacode*}
145
146 % =====
147 % Internal data structures
148 % =====
149
150 \seq_new:N \g_mvar_names_seq      % all model variable names
151 \seq_new:N \g_mvar_start_seq      % names with an initial value (for the table)
152 \seq_new:N \g_mrul_seq           % model rules (display strings)
153 \seq_new:N \g_mrul_type_seq      % type per display row: rule | cont
154 \seq_new:N \g_mrul_calc_seq      % model rules for execution: {name}{expr}
155 \int_new:N \g_mrul_counter_int    % rule counter (display numbering)
156 \tl_new:N \g_numodel_stop_expr_tl % stop-condition expression for execution
157 \int_new:N \g_numodel_steps_int   % number of executed steps
158 \int_new:N \g_numodel_maxiter_int % safety limit (init via \numodelsetup)
159 \int_new:N \g_numodel_gridmaxx_int % \graphicmodel wrap threshold (0 = off)
160
161 % --- Graphic-model infrastructure (Phase 1) ---
162 \tl_new:N \l__numodel_tmp_tl      % temporary helper
163 \tl_new:N \l__numodel_scratch_tl  % scratch for type/text checks
164 \tl_new:N \l__numodel_scratch_y_tl % scratch y-coord (svg_y lookup)
165
166 % Diagram style for \graphicmodel (see \numodelsetup):

```

```

167 %   tight      -- valve takes the label of the direct inflow variable;
168 %               the aux/const is moved to the valve position and not
169 %               drawn as a separate node (compact, default).
170 %   forrester -- valve has no label; the aux/const stays at its own
171 %               gridy position; causal arrow from aux/const to the
172 %               valve.
173 %   edu         -- combination: valve takes the label *and* a separate
174 %               aux/const node remains with a causal arrow to it.
175 \tl_new:N \g__numodel_diagram_style_tl
176
177 % Sub-keys that fine-tune the appearance of the Forrester diagram
178 % (see \numodelsetup). Empty value = "follow diagram-style default".
179 %   flowarrow-style : hollow | filled
180 %               forrester -> hollow, tight|edu -> filled
181 %   valve-style    : valve | circle | edu
182 %               forrester -> valve, tight|edu -> edu
183 %   flowarrow-cloud-tip: true | false
184 %               forrester -> true, tight|edu -> false
185 % Per-variable override for flowarrow-cloud-tip is stored in
186 % \<name>flowcloud (set via the [flowarrow-cloud-tip=...] key on
187 % \mvar; empty means "follow global key").
188 \tl_new:N \g__numodel_flowarrow_style_tl
189 \tl_new:N \g__numodel_valve_style_tl
190 \tl_new:N \g__numodel_flowcloud_tl
191
192 % Units in the \textmodel initial-values column (see \numodelsetup):
193 %   true  (default) -- initial value rendered via \<name>qty
194 %               (number + unit)
195 %   false -- initial value rendered via \<name>num
196 %               (number only)
197 % Per-table override via \textmodel[units=true|false].
198 \bool_new:N \g__numodel_units_bool
199
200 % tabularray environment used by \textmodel (see \numodelsetup{tblrenv}):
201 %   longtblr -- default; page-breakable; suitable for body text.
202 %   tblr      -- basic; no page breaks; choose this when \textmodel sits
203 %               inside an outer environment that already suppresses page
204 %               breaks (subfigure, minipage, ...).
205 %   talltblr -- like tblr (no page breaks) but supports captions/notes.
206 % Per-table override via \textmodel[tblrenv=...].
207 \tl_new:N \g__numodel_tblrenv_tl
208
209 % Decimal separator for \textmodel initial values and \diagrammodel
210 % tick values (see \numodelsetup{decimal-separator}):
211 %   point -- '.' (siunitx output-decimal-marker={.})
212 %   comma -- ',' (siunitx output-decimal-marker={,})
213 % The default follows syntax: english -> point, coachtaal -> comma.
214 % An explicit decimal-separator key sets
215 % \g__numodel_dsep_explicit_bool so that a later syntax change no
216 % longer overrules the choice.
217 \tl_new:N \g__numodel_dsep_tl
218 \bool_new:N \g__numodel_dsep_explicit_bool
219
220 % =====
221 % Syntax lookup
222 % =====
223 % Determines the language of the text-rendered model. Affects display
224 % only (\textmodel, \mruletext, \mstop); \computemodel always uses
225 % \fpeval internally and is language-agnostic. Each value of
226 % \g__numodel_syntax_tl is a tag that selects both the backing
227 % \__numodel_kw_<tag>_<key>: macros and the file numodel-<tag>.def
228 % that defines them; the package ships EN (English/XMILE) and NL
229 % (Dutch/CoachTaal).
230 %
231 % The CTAN default is EN. The initial value is set below via
232 % \numodelsetup.

```

```

233
234 \tl_new:N \g__numodel_syntax_tl
235
236 % Lookup (fully expandable): \__numodel_kw:n {<key>} returns the
237 % translation in the current syntax language. The key also controls
238 % the surrounding spaces (see keys 'then' vs 'then_nl' etc.). The
239 % backing macros \__numodel_kw_<LANG>_<key>: are provided by the
240 % language file numodel-<LANG>.def loaded by \__numodel_load_syntax_def:n
241 % (see below).
242 \cs_new:Npn \__numodel_kw:n #1
243 { \use:c { __numodel_kw_ \g__numodel_syntax_tl _ #1 : } }
244
245 % Pre-computed translations in tl variables so they are usable via
246 % \u{} in l3regex replacement text. (l3regex treats {...} in the
247 % replacement as brace groups, so a bare \__numodel_kw:n{key} call
248 % fails there.)
249 \tl_new:N \g__numodel_kw_if_tl
250 \tl_new:N \g__numodel_kw_then_tl
251 \tl_new:N \g__numodel_kw_then_nl_tl
252 \tl_new:N \g__numodel_kw_else_tl
253 \tl_new:N \g__numodel_kw_else_nl_tl
254 \tl_new:N \g__numodel_kw_endif_tl
255 \tl_new:N \g__numodel_kw_endif_nl_tl
256 \tl_new:N \g__numodel_kw_and_tl
257 \tl_new:N \g__numodel_kw_or_tl
258 \tl_new:N \g__numodel_kw_not_tl
259 \tl_new:N \g__numodel_kw_sign_tl
260 \tl_new:N \g__numodel_kw_abs_tl
261 \tl_new:N \g__numodel_kw_sqrt_tl
262 \tl_new:N \g__numodel_kw_exp_tl
263 \tl_new:N \g__numodel_kw_ln_tl
264 \tl_new:N \g__numodel_kw_sin_tl
265 \tl_new:N \g__numodel_kw_cos_tl
266 \tl_new:N \g__numodel_kw_tan_tl
267 \tl_new:N \g__numodel_kw_asin_tl
268 \tl_new:N \g__numodel_kw_acos_tl
269 \tl_new:N \g__numodel_kw_stop_tl
270
271 % Recomputes all tl caches from \g__numodel_syntax_tl. Must be
272 % called after any change to the syntax language.
273 \cs_new_protected:Npn \__numodel_refresh_kw:
274 {
275   \tl_gset:Ne \g__numodel_kw_if_tl      { \__numodel_kw:n {if}      }
276   \tl_gset:Ne \g__numodel_kw_then_tl    { \__numodel_kw:n {then}    }
277   \tl_gset:Ne \g__numodel_kw_then_nl_tl { \__numodel_kw:n {then_nl}  }
278   \tl_gset:Ne \g__numodel_kw_else_tl    { \__numodel_kw:n {else}    }
279   \tl_gset:Ne \g__numodel_kw_else_nl_tl { \__numodel_kw:n {else_nl}  }
280   \tl_gset:Ne \g__numodel_kw_endif_tl   { \__numodel_kw:n {endif}   }
281   \tl_gset:Ne \g__numodel_kw_endif_nl_tl { \__numodel_kw:n {endif_nl} }
282   \tl_gset:Ne \g__numodel_kw_and_tl     { \__numodel_kw:n {and}     }
283   \tl_gset:Ne \g__numodel_kw_or_tl      { \__numodel_kw:n {or}      }
284   \tl_gset:Ne \g__numodel_kw_not_tl     { \__numodel_kw:n {not}     }
285   \tl_gset:Ne \g__numodel_kw_sign_tl    { \__numodel_kw:n {sign}    }
286   \tl_gset:Ne \g__numodel_kw_abs_tl     { \__numodel_kw:n {abs}     }
287   \tl_gset:Ne \g__numodel_kw_sqrt_tl    { \__numodel_kw:n {sqrt}    }
288   \tl_gset:Ne \g__numodel_kw_exp_tl     { \__numodel_kw:n {exp}     }
289   \tl_gset:Ne \g__numodel_kw_ln_tl      { \__numodel_kw:n {ln}      }
290   \tl_gset:Ne \g__numodel_kw_sin_tl     { \__numodel_kw:n {sin}     }
291   \tl_gset:Ne \g__numodel_kw_cos_tl     { \__numodel_kw:n {cos}     }
292   \tl_gset:Ne \g__numodel_kw_tan_tl     { \__numodel_kw:n {tan}     }
293   \tl_gset:Ne \g__numodel_kw_asin_tl    { \__numodel_kw:n {asin}    }
294   \tl_gset:Ne \g__numodel_kw_acos_tl    { \__numodel_kw:n {acos}    }
295   \tl_gset:Ne \g__numodel_kw_stop_tl    { \__numodel_kw:n {stop}    }
296   % tabularray continuation markers (used by longtblr in \textmodel)
297   \tl_gset:Ne \tblrcontfootname { \__numodel_kw:n {contfoot} }
298   \tl_gset:Ne \tblrcontheadname { \__numodel_kw:n {conthead} }

```



```

299 }
300
301 % Helper: \_numodel_kwt:n {<key>} expands to \text{<kw-value>}.
302 % Meant for use inside \tl_set:Nx / \seq_gput_right:Nx -- the kw is
303 % inserted during e-expansion while \text remains protected.
304 \cs_new:Npn \_numodel_kwt:n #1 { \text { \_numodel_kw:n {#1} } }
305
306 % Language-file loader. Each tag <LANG> is backed by a file
307 % numodel-<LANG>.def installed in a kpse-searched directory; the
308 % package ships numodel-EN.def and numodel-NL.def, users can drop
309 % additional files in TEXMFHOME/tex/latex/numodel/ without rebuilding
310 % the package.
311 %
312 % A small alias property maps the historical long names to the
313 % canonical two-letter tag used in the file name and the internal
314 % macro names. Users who add their own file just pass the file's tag
315 % to syntax=<tag> directly; no alias is required.
316 \prop_new:N \g__numodel_syntax_aliases_prop
317 \prop_gput:Nnn \g__numodel_syntax_aliases_prop { english } { EN }
318 \prop_gput:Nnn \g__numodel_syntax_aliases_prop { coachtaal } { NL }
319 \prop_gput:Nnn \g__numodel_syntax_aliases_prop { dutch } { NL }
320
321 % Tags whose .def file has already been input during this run, so the
322 % lookup only triggers \InputIfFileExists the first time.
323 \seq_new:N \g__numodel_loaded_syntax_seq
324
325 \msg_new:nnn { numodel } { unknown-syntax }
326 {
327   Cannot-load-syntax~'#1':~file~'numodel-#1.def'~not~found~by~
328   kpse.~The~package~ships~with~EN~(English)~and~NL~(Dutch~
329   CoachTaal);~drop~your~own~numodel-<LANG>.def~in~
330   TEXMFHOME/tex/latex/numodel/~to~add~more~languages.
331 }
332
333 % \_numodel_load_syntax_def:n {<tag>}
334 % Inputs numodel-<tag>.def the first time it is requested. Raises a
335 % LaTeX error if the file is not found by kpse.
336 \cs_new_protected:Npn \_numodel_load_syntax_def:n #1
337 {
338   \seq_if_in:NnF \g__numodel_loaded_syntax_seq {#1}
339   {
340     \InputIfFileExists { numodel-#1.def }
341     { \seq_gput_right:Nn \g__numodel_loaded_syntax_seq {#1} }
342     { \msg_error:nnn { numodel } { unknown-syntax } {#1} }
343   }
344 }
345
346 \tl_new:N \l__numodel_syntax_tag_tl
347
348 % \_numodel_set_syntax:n {<value>}
349 % Public-facing setter behind the syntax key. Resolves aliases,
350 % loads the matching .def file (once), publishes the canonical tag
351 % in \g__numodel_syntax_tl, refreshes the keyword cache, and -- when
352 % the user has not pinned a decimal separator explicitly -- adopts the
353 % language's preferred decimal mark via the optional
354 % \_numodel_kw_<LANG>dsep_default: hook.
355 \cs_new_protected:Npn \_numodel_set_syntax:n #1
356 {
357   \prop_get:NnNF \g__numodel_syntax_aliases_prop {#1}
358   \l__numodel_syntax_tag_tl
359   { \tl_set:Nn \l__numodel_syntax_tag_tl {#1} }
360   \exp_args:NV \_numodel_load_syntax_def:n \l__numodel_syntax_tag_tl
361   \tl_gset_eq:NN \g__numodel_syntax_tl \l__numodel_syntax_tag_tl
362   \_numodel_refresh_kw:
363   \bool_if:NF \g__numodel_dsep_explicit_bool
364   {

```

```

365 \cs_if_exist:cTF
366 { __numodel_kw_ \g__numodel_syntax_tl _dsep_default: }
367 { \tl_gset:Ne \g__numodel_dsep_tl { \__numodel_kw:n {dsep_default} } }
368 { \tl_gset:Nn \g__numodel_dsep_tl { point } }
369 }
370 }
371
372 % Applies the decimal separator (only for the duration of the
373 % surrounding TeX group, so a document-wide \sisetup is left
374 % untouched). Calls both siunitx (for \num/\qty in \textmodel
375 % initial values) and pgfplots' tick styles (for \diagrammodel tick
376 % labels) so the choice is consistent everywhere.
377 \cs_new_protected:Npn \__numodel_apply_dsep:
378 {
379   \str_if_eq:VnTF \g__numodel_dsep_tl { comma }
380   {
381     \sisetup{ output-decimal-marker = {,} }
382     % Append behind the existing numodel/axis style so our
383     % xticklabel-style replaces what numodel-plot hard-codes.
384     % The \pgfplotsset mutation is \def-based and hence
385     % group-local.
386     \pgfplotsset
387     {
388       numodel/axis/.append-style=
389       {
390         xticklabel-style=
391         { /pgf/number-format/.cd, use-comma, /tikz/.cd } ,
392         yticklabel-style=
393         { /pgf/number-format/.cd, use-comma, /tikz/.cd } ,
394       }
395     }
396   }
397   {
398     \sisetup{ output-decimal-marker = {.} }
399     \pgfplotsset
400     {
401       numodel/axis/.append-style=
402       {
403         xticklabel-style=
404         { /pgf/number-format/.cd, use-period, /tikz/.cd } ,
405         yticklabel-style=
406         { /pgf/number-format/.cd, use-period, /tikz/.cd } ,
407       }
408     }
409   }
410 }
411
412 % =====
413 % Configuration command \numodelsetup
414 % =====
415 % Runtime API for settings. Keys:
416 % syntax      -- language tag (file numodel-<tag>.def must be on kpse
417 %              path). Built-in: EN, NL. Legacy aliases: english,
418 %              coachtaal, dutch.
419 % maxiter     -- safety limit for \computemodel (default 20000)
420 % graphscalex -- horizontal grid spacing \dgridx for Forrester
421 %              diagrams (default 2)
422 % graphscaley -- vertical grid spacing \dgridy for Forrester
423 %              diagrams (default 2)
424
425 % Helper for choice-with-empty-reset: validates #4 against #3 (a
426 % comma-separated whitelist) and writes it into #1 (a tl). An empty
427 % value clears the tl (which means "follow the diagram-style
428 % default" for the flowarrow-style / valve-style /
429 % flowarrow-cloud-tip keys). Any other value triggers a warning.
430 \msg_new:nnn { numodel } { bad-choice }

```

```

431 { Key~'#1'~accepts-only~#2,~or~empty~to~reset.~Got:~'#3'. }
432
433 \cs_new_protected:Npn \__numodel_setup_choice:Nnnn #1 #2 #3 #4
434 {
435   \tl_if_blank:nTF {#4}
436   { \tl_gclear:N #1 }
437   {
438     \clist_if_in:nnTF {#3} {#4}
439     { \tl_gset:Nn #1 {#4} }
440     { \msg_warning:nnnnn { numodel } { bad-choice } {#2} {#3} {#4} }
441   }
442 }
443
444 % Local-tl variant: same validation, local set/clear. Used by the
445 % \graphicmodel one-render override and the per-\mvar override.
446 \cs_new_protected:Npn \__numodel_local_choice:Nnnn #1 #2 #3 #4
447 {
448   \tl_if_blank:nTF {#4}
449   { \tl_clear:N #1 }
450   {
451     \clist_if_in:nnTF {#3} {#4}
452     { \tl_set:Nn #1 {#4} }
453     { \msg_warning:nnnnn { numodel } { bad-choice } {#2} {#3} {#4} }
454   }
455 }
456
457 \keys_define:nn { numodel / setup }
458 {
459   syntax .code:n =
460   { \__numodel_set_syntax:n {#1} },
461   maxiter .int_gset:N = \g_numodel_maxiter_int,
462   graphscalex .code:n = { \tl_gset:Nn \dgridx {#1} },
463   graphscaley .code:n = { \tl_gset:Nn \dgridy {#1} },
464   gridmaxx .int_gset:N = \g_numodel_gridmaxx_int,
465   diagram-style .choice:,
466   diagram-style / tight .code:n =
467   { \tl_gset:Nn \g__numodel_diagram_style_tl { tight } },
468   diagram-style / forrester .code:n =
469   { \tl_gset:Nn \g__numodel_diagram_style_tl { forrester } },
470   diagram-style / edu .code:n =
471   { \tl_gset:Nn \g__numodel_diagram_style_tl { edu } },
472   flowarrow-style .code:n =
473   { \__numodel_setup_choice:Nnnn \g__numodel_flowarrow_style_tl
474     { flowarrow-style } { hollow , filled } {#1} },
475   valve-style .code:n =
476   { \__numodel_setup_choice:Nnnn \g__numodel_valve_style_tl
477     { valve-style } { valve , circle , edu } {#1} },
478   flowarrow-cloud-tip .code:n =
479   { \__numodel_setup_choice:Nnnn \g__numodel_flowcloud_tl
480     { flowarrow-cloud-tip } { true , false } {#1} },
481   units .choice:,
482   units / true .code:n =
483   { \bool_gset_true:N \g__numodel_units_bool },
484   units / false .code:n =
485   { \bool_gset_false:N \g__numodel_units_bool },
486   tblrenv .choice:,
487   tblrenv / tblr .code:n =
488   { \tl_gset:Nn \g__numodel_tblrenv_tl { tblr } },
489   tblrenv / longtblr .code:n =
490   { \tl_gset:Nn \g__numodel_tblrenv_tl { longtblr } },
491   tblrenv / talltblr .code:n =
492   { \tl_gset:Nn \g__numodel_tblrenv_tl { talltblr } },
493   decimal-separator .choice:,
494   decimal-separator / comma .code:n =
495   {
496     \tl_gset:Nn \g__numodel_dsep_tl { comma }

```

```

497     \bool_gset_true:N \g__numodel_dsep_explicit_bool
498   },
499   decimal-separator / point .code:n =
500   {
501     \tl_gset:Nn \g__numodel_dsep_tl { point }
502     \bool_gset_true:N \g__numodel_dsep_explicit_bool
503   },
504 }
505
506 \NewDocumentCommand{\numodelsetup}{ m }
507 { \keys_set:nn { numodel / setup } {#1} }
508
509 % Defaults (this call also initialises \g__numodel_syntax_tl,
510 % \g__numodel_maxiter_int, \dgridx, \dgridy and
511 % \g__numodel_diagram_style_tl).
512 \numodelsetup
513 {
514   syntax          = EN,
515   maxiter         = 20000,
516   graphscalex     = 2,
517   graphscaley     = 2,
518   diagram-style   = tight,
519   units           = true,
520   tblrenv         = longtblr,
521 }
522
523 % Package-time options. \usepackage[syntax=NL]{numodel} delegates
524 % to the same key infrastructure as \numodelsetup.
525 \keys_define:nn { numodel / pkg }
526 {
527   syntax          .meta:nn = { numodel / setup }{ syntax          = #1 },
528   maxiter         .meta:nn = { numodel / setup }{ maxiter         = #1 },
529   graphscalex     .meta:nn = { numodel / setup }{ graphscalex     = #1 },
530   graphscaley     .meta:nn = { numodel / setup }{ graphscaley     = #1 },
531   gridmaxx        .meta:nn = { numodel / setup }{ gridmaxx        = #1 },
532   diagram-style   .meta:nn = { numodel / setup }{ diagram-style   = #1 },
533   flowarrow-style .meta:nn = { numodel / setup }{ flowarrow-style = #1 },
534   valve-style     .meta:nn = { numodel / setup }{ valve-style     = #1 },
535   flowarrow-cloud-tip .meta:nn = { numodel / setup }{ flowarrow-cloud-tip = #1 },
536   units           .meta:nn = { numodel / setup }{ units           = #1 },
537   tblrenv         .meta:nn = { numodel / setup }{ tblrenv         = #1 },
538   decimal-separator .meta:nn = { numodel / setup }{ decimal-separator = #1 },
539 }
540 \ProcessKeyOptions [ numodel / pkg ]
541
542 % =====
543 % Prefix system
544 % =====
545 % Each model lives under a prefix (a short string). The "live state"
546 % sits in the \g_mvar_*, \g_mrul_*, \g_numodel_* variables above;
547 % \newmodelprefix and \switchmodelprefix swap that state to/from
548 % per-prefix backup storage.
549 %
550 % - \g_numodel_current_prefix_tl: current prefix (empty at package load)
551 % - \g_numodel_prefixes_seq: list of all registered prefixes
552 % - \l__numodel_cmd_prefix_tl: prefix passed via [prefix=...] key
553 % - \l__numodel_eff_prefix_tl: effective prefix for the current command
554
555 \tl_new:N \g_numodel_current_prefix_tl
556 \seq_new:N \g_numodel_prefixes_seq
557 \tl_new:N \l__numodel_cmd_prefix_tl
558 \tl_new:N \l__numodel_eff_prefix_tl
559 \tl_new:N \l__numodel_fullname_tl
560
561 % Per-prefix storage: on swap the current state is copied to
562 % \g__numodel_<P>_<slot>_{seq,tl,int,prop}. Load goes the other way.

```

```

563 \cs_new_protected:Npn \__numodel_save_state:n #1
564 {
565   \seq_gset_eq:cN { g__numodel_ #1 _vars_seq      } \g_mvar_names_seq
566   \seq_gset_eq:cN { g__numodel_ #1 _starts_seq    } \g_mvar_start_seq
567   \seq_gset_eq:cN { g__numodel_ #1 _rules_seq     } \g_mrulerule_seq
568   \seq_gset_eq:cN { g__numodel_ #1 _ruletypes_seq } \g_mrulerule_type_seq
569   \seq_gset_eq:cN { g__numodel_ #1 _rulecalc_seq  } \g_mrulerule_calc_seq
570   \int_gset:cn { g__numodel_ #1 _rulecounter_int }
571   { \int_use:N \g_mrulerule_counter_int }
572   \tl_gset_eq:cN { g__numodel_ #1 _stopexpr_tl    } \g_numodel_stop_expr_tl
573   \int_gset:cn { g__numodel_ #1 _steps_int       }
574   { \int_use:N \g_numodel_steps_int }
575 }
576
577 \cs_new_protected:Npn \__numodel_load_state:n #1
578 {
579   \seq_gset_eq:Nc \g_mvar_names_seq { g__numodel_ #1 _vars_seq      }
580   \seq_gset_eq:Nc \g_mvar_start_seq { g__numodel_ #1 _starts_seq    }
581   \seq_gset_eq:Nc \g_mrulerule_seq  { g__numodel_ #1 _rules_seq     }
582   \seq_gset_eq:Nc \g_mrulerule_type_seq { g__numodel_ #1 _ruletypes_seq }
583   \seq_gset_eq:Nc \g_mrulerule_calc_seq { g__numodel_ #1 _rulecalc_seq  }
584   \int_gset:Nn \g_mrulerule_counter_int
585   { \int_use:c { g__numodel_ #1 _rulecounter_int } }
586   \tl_gset_eq:Nc \g_numodel_stop_expr_tl { g__numodel_ #1 _stopexpr_tl }
587   \int_gset:Nn \g_numodel_steps_int
588   { \int_use:c { g__numodel_ #1 _steps_int } }
589 }
590
591 \cs_new_protected:Npn \__numodel_clear_state:
592 {
593   \seq_gclear:N \g_mvar_names_seq
594   \seq_gclear:N \g_mvar_start_seq
595   \seq_gclear:N \g_mrulerule_seq
596   \seq_gclear:N \g_mrulerule_type_seq
597   \seq_gclear:N \g_mrulerule_calc_seq
598   \int_gzero:N \g_mrulerule_counter_int
599   \tl_gclear:N \g_numodel_stop_expr_tl
600   \int_gzero:N \g_numodel_steps_int
601 }
602
603 % Initialise per-prefix backup variables (once, at \newmodelprefix).
604 \cs_new_protected:Npn \__numodel_init_backup:n #1
605 {
606   \seq_new:c { g__numodel_ #1 _vars_seq      }
607   \seq_new:c { g__numodel_ #1 _starts_seq    }
608   \seq_new:c { g__numodel_ #1 _rules_seq     }
609   \seq_new:c { g__numodel_ #1 _ruletypes_seq }
610   \seq_new:c { g__numodel_ #1 _rulecalc_seq  }
611   \int_new:c { g__numodel_ #1 _rulecounter_int }
612   \tl_new:c { g__numodel_ #1 _stopexpr_tl    }
613   \int_new:c { g__numodel_ #1 _steps_int     }
614 }
615
616 % Public commands for prefix management
617 \NewDocumentCommand{\newmodelprefix}{m}
618 {
619   \typeout{NEWPREFIX~start:~#1}
620   \seq_if_in:NnTF \g_numodel_prefixes_seq {#1}
621   { \msg_error:nnn { numodel } { prefix-exists } {#1} }
622   {
623     \typeout{NEWPREFIX~save~current:~\g_numodel_current_prefix_tl}
624     % Save current state under the previous prefix (if any)
625     \tl_if_empty:NF \g_numodel_current_prefix_tl
626     { \exp_args:NV \__numodel_save_state:n \g_numodel_current_prefix_tl }
627     \typeout{NEWPREFIX~register}
628     % Register new prefix + init backup vars + Lua state

```

```

629     \seq_gput_right:Nn \g_numodel_prefixes_seq {#1}
630     \typeout{NEWPREFIX~init_backup}
631     \__numodel_init_backup:n {#1}
632     \typeout{NEWPREFIX~lua_init}
633     \directlua{ numodel.init_prefix("#1") }
634     \typeout{NEWPREFIX~clear_state}
635     % Clear current state and set the prefix
636     \__numodel_clear_state:
637     \typeout{NEWPREFIX~set_current}
638     \tl_gset:Nn \g_numodel_current_prefix_tl {#1}
639     \typeout{NEWPREFIX~done:~#1}
640   }
641 }
642
643 \NewDocumentCommand{\switchmodelprefix}{ m }
644 {
645   \seq_if_in:NnTF \g_numodel_prefixes_seq {#1}
646   {
647     % Save current state (if any), load the new one
648     \tl_if_empty:NF \g_numodel_current_prefix_tl
649     { \exp_args:NV \__numodel_save_state:n \g_numodel_current_prefix_tl }
650     \__numodel_load_state:n {#1}
651     \tl_gset:Nn \g_numodel_current_prefix_tl {#1}
652   }
653   { \msg_error:nnn { numodel } { prefix-unknown } {#1} }
654 }
655
656 % Iterate over *all* variables from *all* prefixes.
657 % #1 = code that uses ##1 = full variable name (prefix + short).
658 % Public registration API for external tools such as worksheet.tex.
659 \NewDocumentCommand{\NumodelForEachVar}{ +m }
660 {
661   \seq_map_inline:Nn \g_numodel_prefixes_seq
662   {
663     \seq_map_inline:cn { g__numodel_ ##1 _vars_seq } {#1}
664   }
665 }
666
667 % Resolve the effective prefix for a command call. Takes
668 % [prefix=<p>] from the keyval argument; otherwise falls back to the
669 % current prefix. Stores the result in \l_numodel_eff_prefix_tl.
670 \keys_define:nn { numodel / cmd }
671 {
672   prefix .tl_set:N = \l__numodel_cmd_prefix_tl,
673   % \graphicmodel-only: temporary override of the global
674   % diagram-style. Cleared after every \graphicmodel call.
675   diagram-style .choice:,
676   diagram-style / tight .code:n =
677   { \tl_set:Nn \l__numodel_cmd_diagstyle_tl { tight } },
678   diagram-style / forrester .code:n =
679   { \tl_set:Nn \l__numodel_cmd_diagstyle_tl { forrester } },
680   diagram-style / edu .code:n =
681   { \tl_set:Nn \l__numodel_cmd_diagstyle_tl { edu } },
682   % \graphicmodel-only: temporary overrides of the corresponding
683   % global flowarrow/valve/cloud keys. Empty after every call.
684   flowarrow-style .code:n =
685   { \__numodel_local_choice:Nnnn \l__numodel_cmd_flowarrow_tl
686     { flowarrow-style } { hollow , filled } {#1} },
687   valve-style .code:n =
688   { \__numodel_local_choice:Nnnn \l__numodel_cmd_valve_tl
689     { valve-style } { valve , circle , edu } {#1} },
690   flowarrow-cloud-tip .code:n =
691   { \__numodel_local_choice:Nnnn \l__numodel_cmd_flowcloud_tl
692     { flowarrow-cloud-tip } { true , false } {#1} },
693   % \textmodel-only: temporary override of the global units bool.
694   % Cleared after every \textmodel call.

```

```

695     units                                .choice:,
696     units / true                         .code:n =
697     { \tl_set:Nn \l__numodel_cmd_units_tl { true } },
698     units / false                        .code:n =
699     { \tl_set:Nn \l__numodel_cmd_units_tl { false } },
700     % \textmodel-only: temporary override of the global tblrenv tl.
701     % Cleared after every \textmodel call.
702     tblrenv                              .choice:,
703     tblrenv / tblr                       .code:n =
704     { \tl_set:Nn \l__numodel_cmd_tblrenv_tl { tblr } },
705     tblrenv / longtblr                   .code:n =
706     { \tl_set:Nn \l__numodel_cmd_tblrenv_tl { longtblr } },
707     tblrenv / talltblr                   .code:n =
708     { \tl_set:Nn \l__numodel_cmd_tblrenv_tl { talltblr } },
709 }
710 \tl_new:N \l__numodel_cmd_diagstyle_tl
711 \tl_new:N \l__numodel_cmd_flowarrow_tl
712 \tl_new:N \l__numodel_cmd_valve_tl
713 \tl_new:N \l__numodel_cmd_flowcloud_tl
714 \tl_new:N \l__numodel_cmd_units_tl
715 \tl_new:N \l__numodel_cmd_tblrenv_tl
716
717 \cs_new_protected:Npn \__numodel_resolve_prefix:n #1
718 {
719     \tl_clear:N \l__numodel_cmd_prefix_tl
720     \keys_set:nn { numodel / cmd } {#1}
721     \tl_if_empty:NTF \l__numodel_cmd_prefix_tl
722     { \tl_set_eq:NN \l__numodel_eff_prefix_tl \g_numodel_current_prefix_tl }
723     { \tl_set_eq:NN \l__numodel_eff_prefix_tl \l__numodel_cmd_prefix_tl }
724 }
725
726 % Execute code under a temporarily different prefix (via state swap).
727 % #1 = target prefix (tl, in variable)
728 % #2 = code
729 \cs_new_protected:Npn \__numodel_with_prefix:Nn #1 #2
730 {
731     \tl_if_eq:NNTF #1 \g_numodel_current_prefix_tl
732     { #2 } % already the current prefix; no swap needed
733     {
734         \tl_set_eq:NN \l__numodel_saved_prefix_tl \g_numodel_current_prefix_tl
735         \exp_args:NV \switchmodelprefix #1
736         #2
737         \exp_args:NV \switchmodelprefix \l__numodel_saved_prefix_tl
738     }
739 }
740 \tl_new:N \l__numodel_saved_prefix_tl
741
742 % Build the full name = <prefix><shortname>. Stored in
743 % \l__numodel_fullname_tl.
744 \cs_new_protected:Npn \__numodel_set_fullname:n #1
745 {
746     \tl_set:Ne \l__numodel_fullname_tl { \l__numodel_eff_prefix_tl #1 }
747 }
748
749 % Keys for the optional argument of \mvar (grid position +
750 % initial-value aliases + prefix).
751 \tl_new:N \l__numodel_alias_tl
752 \tl_new:N \l__numodel_aliasleft_tl
753 \tl_new:N \l__numodel_aliasright_tl
754
755 \keys_define:nn { numodel / mvar }
756 {
757     prefix      .tl_set:N = \l__numodel_cmd_prefix_tl ,
758     gridx       .tl_set:N = \l__numodel_gridx_tl ,
759     gridy       .tl_set:N = \l__numodel_gridy_tl ,
760     alias       .tl_set:N = \l__numodel_alias_tl ,

```

```

761 aliasleft .tl_set:N = \l__numodel_aliasleft_tl ,
762 aliasright .tl_set:N = \l__numodel_aliasright_tl ,
763 flowarrow-cloud-tip .code:n =
764 { \__numodel_local_choice:Nnnn \l__numodel_mvar_flowcloud_tl
765 { flowarrow-cloud-tip } { true , false } {#1} },
766 }
767 \tl_new:N \l__numodel_mvar_flowcloud_tl
768
769 % =====
770 % \mvar[keys]{name}{display}{startvalue}{unit}{sig}{type}
771 % =====
772 % Declare a model variable. Generates the following macros:
773 %
774 % \<name> -- numeric value (\edef + \fpeval), or warning if empty
775 % \<name>text -- display name for the model table (e.g. F_{res})
776 % \<name>unit -- SI unit (e.g. kN)
777 % \<name>unitraw - raw SI unit (e.g. \kilo\N)
778 % \<name>sign -- number of significant figures
779 % \<name>type -- variable type: stock, constant, aux, system
780 % (Dutch synonyms voorraad/constante/hulp/systeem
781 % are normalised to the canonical English form)
782 % \<name>coord -- coordinate list (empty; filled by \computemodel)
783 % \<name>gridx -- x position in the graphic model (-1 = auto)
784 % \<name>gridy -- y position in the graphic model (-1 = auto)
785 % \<name>num -- number with significance (via \num)
786 % \<name>qty -- number + unit (via \qty)
787 % \<name>pre -- engineering-prefix notation (via \qty)
788 % \<name>alias -- replaces the whole initial-value cell
789 % (empty = default)
790 % \<name>aliasleft -- replaces the left symbol in the initial value
791 % (empty = default)
792 % \<name>aliasright -- replaces the right number in the initial value
793 % (empty = default)
794 %
795 % Keys accepted in [#1]:
796 % gridx, gridy -- position in the graphic model
797 % alias -- replace the entire initial-value cell
798 % (in math mode)
799 % aliasleft -- replace just the left symbol
800 % aliasright -- replace just the right value
801 %
802 % The base value is registered in \g_defqty_names_seq so that
803 % \includeimage expands it automatically.
804 %
805 % With an empty start value (#3 blank), the base value is not
806 % numerically defined; using it issues a warning. Useful for
807 % auxiliary variables computed by \mrule.
808 %
809 % Example:
810 % \mvar{modM}{m}{80}{\kg}{2}{constant}
811 % \mvar{modFres}{F_{res}}{}{\N}{2}{aux}
812 % \mvar[aliasright=\cdots]{modX}{x}{0}{\m}{3}{stock}
813 % \mvar[alias={x \text{?}}]{modX}{x}{0}{\m}{3}{stock}
814
815 % Normalise the variable type (sixth \mvar argument) to its
816 % canonical English form. Accepts the canonical names
817 % {stock, constant, aux, system} and the Dutch synonyms
818 % {voorraad, constante, hulp, systeem}. An unknown value is left
819 % as-is and a warning is issued. The result is returned through
820 % \l__numodel_type_tl.
821 \tl_new:N \l__numodel_type_tl
822 \cs_new_protected:Npn \__numodel_normalize_type:n #1
823 {
824 \str_case:nnF {#1}
825 {
826 { stock } { \tl_set:Nn \l__numodel_type_tl { stock } }

```



```

827     { constant } { \tl_set:Nn \l__numodel_type_tl { constant } }
828     { aux      } { \tl_set:Nn \l__numodel_type_tl { aux      } }
829     { system   } { \tl_set:Nn \l__numodel_type_tl { system   } }
830     { voorraad } { \tl_set:Nn \l__numodel_type_tl { stock    } }
831     { constante } { \tl_set:Nn \l__numodel_type_tl { constant } }
832     { hulp     } { \tl_set:Nn \l__numodel_type_tl { aux      } }
833     { systeem  } { \tl_set:Nn \l__numodel_type_tl { system   } }
834   }
835   {
836     \msg_warning:nne { numodel } { unknown-type } {#1}
837     \tl_set:Nn \l__numodel_type_tl {#1}
838   }
839 }
840
841 \makeatletter
842 \NewDocumentCommand{\mvar}{ O{} m m m m m m }{%
843   \typeout{MVAR~start:~#2}
844   % Parse optional keys (prefix, gridx, gridy, alias, aliasleft, aliasright,
845   % flowarrow-cloud-tip)
846   \tl_set:Nn \l__numodel_gridx_tl { -1 }
847   \tl_set:Nn \l__numodel_gridy_tl { -1 }
848   \tl_clear:N \l__numodel_alias_tl
849   \tl_clear:N \l__numodel_aliasleft_tl
850   \tl_clear:N \l__numodel_aliasright_tl
851   \tl_clear:N \l__numodel_cmd_prefix_tl
852   \tl_clear:N \l__numodel_mvar_flowcloud_tl
853   \typeout{MVAR~before-keys-set}
854   \keys_set:nn { numodel / mvar } {#1}
855   \typeout{MVAR~after-keys-set}
856   \__numodel_resolve_eff_prefix:
857   \typeout{MVAR~eff:~\l__numodel_eff_prefix_tl}
858   \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
859   { \__numodel_mvar_body:nnnnnn {#2}{#3}{#4}{#5}{#6}{#7} }%
860   \typeout{MVAR~done:~#2}
861 }
862 \makeatother
863
864 % Resolve the effective prefix from \l__numodel_cmd_prefix_tl.
865 \cs_new_protected:Npn \__numodel_resolve_eff_prefix:
866 {
867   \tl_if_empty:NTF \l__numodel_cmd_prefix_tl
868     { \tl_set_eq:NN \l__numodel_eff_prefix_tl \g_numodel_current_prefix_tl }
869     { \tl_set_eq:NN \l__numodel_eff_prefix_tl \l__numodel_cmd_prefix_tl }
870 }
871
872 % The \mvar body runs in the context of the right prefix (live state
873 % has already been swapped).
874 \cs_new_protected:Npn \__numodel_mvar_body:nnnnnn #1 #2 #3 #4 #5 #6
875 {
876   % Guard: current prefix must not be empty (\newmodelprefix required)
877   \tl_if_empty:NT \g_numodel_current_prefix_tl
878     { \msg_error:nn { numodel } { no-prefix } }
879   % Full name = current prefix + short name (#1)
880   \tl_set:Nn \l__numodel_fullname_tl { \g_numodel_current_prefix_tl #1 }
881   \typeout{MVAR~body~fullname:~\l__numodel_fullname_tl}
882   % Register in the per-prefix seq. The defqty registration is
883   % optional: it only fires when the user has loaded the project-
884   % specific 'defqty' system (used for worksheet expansion).
885   % Standalone, the package works without that seq.
886   \seq_gput_right:NV \g_mvar_names_seq \l__numodel_fullname_tl
887   \cs_if_exist:NT \g_defqty_names_seq
888     { \seq_gput_right:NV \g_defqty_names_seq \l__numodel_fullname_tl }
889   \directlua{ numodel.register(
890     "\g_numodel_current_prefix_tl",
891     "\l__numodel_fullname_tl" ) }
892   \cs_if_exist:cT { \l__numodel_fullname_tl }

```

```

893     { \msg_warning:nne { numodel } { redef } { \l__numodel_fullname_tl } }
894 \tl_if_blank:nTF {#3}
895     { \cs_gset:cpe { \l__numodel_fullname_tl } { \fp_eval:n { 0 } } }
896     {
897         \cs_gset:cpe { \l__numodel_fullname_tl } { \fp_eval:n {#3} }
898         \seq_gput_right:N \g_mvar_start_seq \l__numodel_fullname_tl
899     }
900 \cs_gset:cpn { \l__numodel_fullname_tl text } {#2}
901 \cs_gset:cpn { \l__numodel_fullname_tl unit } { \unit{#4} }
902 \cs_gset:cpn { \l__numodel_fullname_tl unitraw } {#4}
903 \cs_gset:cpn { \l__numodel_fullname_tl sign } {#5}
904 \__numodel_normalize_type:n {#6}
905 \cs_gset:cpe { \l__numodel_fullname_tl type }
906     { \tl_use:N \l__numodel_type_tl }
907 \cs_gset:cpn { \l__numodel_fullname_tl min } { inf }
908 \cs_gset:cpn { \l__numodel_fullname_tl max } { -inf }
909 \cs_gset:cpe { \l__numodel_fullname_tl gridx } { \tl_use:N \l__numodel_gridx_tl }
910 \cs_gset:cpe { \l__numodel_fullname_tl gridy } { \tl_use:N \l__numodel_gridy_tl }
911 % Preserve the original user input so that \__numodel_build_graphic:
912 % can reset auto-placed positions to -1 on a second invocation.
913 \cs_gset:cpe { \l__numodel_fullname_tl gridxinit } { \tl_use:N \l__numodel_gridx_tl }
914 \cs_gset:cpe { \l__numodel_fullname_tl gridyinit } { \tl_use:N \l__numodel_gridy_tl }
915 % Pillar A - Lua-side meta for compute_layout (additive in A1).
916 % Detokenize text so that \luaescapestring works on bare chars.
917 \tl_set:Nx \l__numodel_scratch_tl { \detokenize {#2} }
918 \directlua{ numodel.set_meta(
919     "\g_numodel_current_prefix_tl",
920     "\l__numodel_fullname_tl",
921     { type = "\tl_use:N \l__numodel_type_tl",
922       text = "\luaescapestring{\l__numodel_scratch_tl}",
923       gridx = \tl_use:N \l__numodel_gridx_tl,
924       gridy = \tl_use:N \l__numodel_gridy_tl }) }
925 \cs_gset:cpe { \l__numodel_fullname_tl alias } { \exp_not:V \l__numodel_alias_tl }
926 \cs_gset:cpe { \l__numodel_fullname_tl aliasleft } { \exp_not:V \l__numodel_aliasleft_tl }
927 \cs_gset:cpe { \l__numodel_fullname_tl aliasright } { \exp_not:V \l__numodel_aliasright_tl }
928 \cs_gset:cpe { \l__numodel_fullname_tl flowcloud } { \exp_not:V \l__numodel_mvar_flowcloud_tl }
929 \tl_if_blank:nTF {#3}
930     {
931         \exp_args:N \NumodelDefNumCs \l__numodel_fullname_tl { \fp_eval:n {#3} } {#5}
932         \exp_args:N \NumodelDefQtyCs \l__numodel_fullname_tl { \fp_eval:n {#3} } {#5} {#4}
933         \exp_args:N \NumodelDefPreCs \l__numodel_fullname_tl { \fp_eval:n {#3} } {#5} {#4}
934     }
935 }
936
937 % =====
938 % Warning messages
939 % =====
940
941 \msg_new:nnn { numodel } { redef }
942 { Model~variable~'#1'~is~being~redefined. }
943 \msg_new:nnn { numodel } { empty-use }
944 { Model~variable~'#1'~has~no~start~value~and~is~used~before~assignment. }
945 \msg_new:nnn { numodel } { maxiter }
946 { computemodel~stopped~after~\int_use:N \g_numodel_maxiter_int ~iterations~
947 (safety~limit).~Check~stop~condition. }
948 \msg_new:nnn { numodel } { no-stop }
949 { computemodel:~no~stop~condition~defined.~Use~\token_to_str:N \mstop\space
950 before~\token_to_str:N \computemodel. }
951 \msg_new:nnn { numodel } { prefix-exists }
952 { Model~prefix~'#1'~is~already~registered.~
953 Use~\token_to_str:N \switchmodelprefix\space to~switch~to~an~existing~prefix. }
954 \msg_new:nnn { numodel } { prefix-unknown }
955 { Model~prefix~'#1'~is~not~registered.~
956 Use~\token_to_str:N \newmodelprefix\space to~create~it~first. }
957 \msg_new:nnn { numodel } { no-prefix }
958 { No~current~model~prefix.~

```

```

959 Call~\token_to_str:N \newmodelprefix{<name>}\space before~using~model~commands. }
960 \msg_new:nnn { numodel } { unknown-type }
961 { Unknown~variable~type~'#1'.~
962   Use~one~of~stock,~constant,~aux,~system~
963   (or~the~Dutch~aliases~voorraad,~constante,~hulp,~systeem). }
964 \msg_new:nnn { numodel } { unit-mismatch }
965 { Variable~'#1'~has~a~different~unit~from~the~first~variable~
966   (expected~'#2').~Skipping~it~in~\token_to_str:N \diagrammodel\space
967   so~the~remaining~series~share~a~common~y-axis. }
968
969 % =====
970 % Display helpers
971 % =====
972 % Translate computational expressions into syllabus-style display.
973 %
974 % Automatic translations:
975 % \modXxx      -> display name (via \<name>text)
976 % *            -> \cdot
977 % >= <=        -> \geqslant \leqslant
978 % sign(...)    -> SIGN(...) / Teken(...) in coachtaal
979 % abs(...)     -> ABS(...) / Abs(...) in coachtaal
980 % sqrt(...)    -> SQRT(...) / Sqrt(...) in coachtaal
981 % exp(...)     -> EXP(...) / Exp(...) in coachtaal
982 % ln(...)      -> LN(...) / Ln(...) in coachtaal
983 % sin(...)     -> SIN(...) / Sin(...) in coachtaal
984 % cos(...)     -> COS(...) / Cos(...) in coachtaal
985 % tan(...)     -> TAN(...) / Tan(...) in coachtaal
986 % asin(...)    -> ARCSIN(...) / Arcsin(...) in coachtaal
987 % acos(...)    -> ARCCOS(...) / Arccos(...) in coachtaal
988 % &&           -> AND / EN in coachtaal
989 % ||           -> OR / OF in coachtaal
990 % cond ? a : b -> IF cond THEN ... ELSE ... ENDIF (or coachtaal eq.)
991
992 \tl_new:N \l__numodel_display_tl
993 \tl_new:N \l__numodel_lhs_tl
994 \tl_new:N \l__numodel_rhs_tl
995 \tl_new:N \l__numodel_cond_tl
996 \tl_new:N \l__numodel_true_tl
997 \tl_new:N \l__numodel_false_tl
998
999 \cs_new_protected:Npn \__numodel_vars_to_display:N #1
1000 {
1001   \typeout{VTD-input:~\tl_to_str:N #1}
1002   \typeout{VTD-seq:~\seq_use:Nn \g_mvar_names_seq {||}}
1003   % Variable names -> display names
1004   \seq_map_inline:Nn \g_mvar_names_seq
1005   {
1006     \typeout{VTD-iter:~##1}
1007     \cs_if_exist:cT { ##1 text }
1008     {
1009       \tl_set:Nc \l_tmpa_tl { \use:c { ##1 text } }
1010       \typeout{VTD-replace~\c{##1}~with~\l_tmpa_tl}
1011       \regex_replace_all:nnN { \c{##1} } { \u{l_tmpa_tl} } #1
1012     }
1013   }
1014   % Arithmetic operators
1015   \regex_replace_all:nnN { \* } { \c{cdot} \x{20} } #1
1016   \regex_replace_all:nnN { >= } { \c{geqslant} \x{20} } #1
1017   \regex_replace_all:nnN { <= } { \c{leqslant} \x{20} } #1
1018   % Functions (syllabus notation). Order matters: asin/acos must
1019   % match before sin/cos, otherwise the inner sin/cos gets replaced
1020   % first and the outer arc- prefix is left dangling.
1021   \regex_replace_all:nnN { sign \c{ } }
1022   { \c{text}\cB{\ \u{g__numodel_kw_sign_tl}\cE}\c{ } } #1
1023   \regex_replace_all:nnN { abs \c{ } }
1024   { \c{text}\cB{\ \u{g__numodel_kw_abs_tl}\cE}\c{ } } #1

```

```

1025 \regex_replace_all:nnN { sqrt \ ( }
1026 { \c{text}\cB\{ \u{g__numodel_kw_sqrt_tl}\cE\}( } #1
1027 \regex_replace_all:nnN { exp \ ( }
1028 { \c{text}\cB\{ \u{g__numodel_kw_exp_tl}\cE\}( } #1
1029 \regex_replace_all:nnN { ln \ ( }
1030 { \c{text}\cB\{ \u{g__numodel_kw_ln_tl}\cE\}( } #1
1031 \regex_replace_all:nnN { asin \ ( }
1032 { \c{text}\cB\{ \u{g__numodel_kw_asin_tl}\cE\}( } #1
1033 \regex_replace_all:nnN { acos \ ( }
1034 { \c{text}\cB\{ \u{g__numodel_kw_acos_tl}\cE\}( } #1
1035 \regex_replace_all:nnN { sin \ ( }
1036 { \c{text}\cB\{ \u{g__numodel_kw_sin_tl}\cE\}( } #1
1037 \regex_replace_all:nnN { cos \ ( }
1038 { \c{text}\cB\{ \u{g__numodel_kw_cos_tl}\cE\}( } #1
1039 \regex_replace_all:nnN { \b tan \ ( }
1040 { \c{text}\cB\{ \u{g__numodel_kw_tan_tl}\cE\}( } #1
1041 % Logical operators (syllabus notation)
1042 \regex_replace_all:nnN { \&\& }
1043 { \c{text}\cB\{ \u{g__numodel_kw_and_tl}\cE\} } #1
1044 \regex_replace_all:nnN { \|\| }
1045 { \c{text}\cB\{ \u{g__numodel_kw_or_tl}\cE\} } #1
1046 }
1047
1048 % Ternary detection: cond ? true_expr : false_expr
1049 % Converted to: IF cond THEN lhs = true ELSE lhs = false ENDIF
1050 % (or the coachtaal equivalent). Supports only flat (non-nested)
1051 % ternaries.
1052 \bool_new:N \l__numodel_is_ternary_bool
1053
1054 \cs_new_protected:Npn \__numodel_parse_ternary:nN #1 #2
1055 {
1056   \bool_set_false:N \l__numodel_is_ternary_bool
1057   \regex_match:nnT { (.+) \? (.+) \: (.+) } {#1}
1058   {
1059     \bool_set_true:N \l__numodel_is_ternary_bool
1060     \tl_set:Nn \l__numodel_cond_tl {#1}
1061     \regex_replace_once:nnN { \s*(.+?) \s* \? .+ } { \1 } \l__numodel_cond_tl
1062     \tl_set:Nn \l__numodel_true_tl {#1}
1063     \regex_replace_once:nnN { .+? \? \s* (.+?) \s* \: .+ } { \1 } \l__numodel_true_tl
1064     \tl_set:Nn \l__numodel_false_tl {#1}
1065     \regex_replace_once:nnN { .+ \: \s* (.+?) \s* \Z } { \1 } \l__numodel_false_tl
1066     \__numodel_vars_to_display:N \l__numodel_cond_tl
1067     \__numodel_vars_to_display:N \l__numodel_true_tl
1068     \__numodel_vars_to_display:N \l__numodel_false_tl
1069   }
1070 }
1071
1072 % =====
1073 % \mrule[keys]{varname}{calculation}
1074 % =====
1075 % Declare a model rule.
1076 %
1077 % #1 (star)      -- starred variant: multiline IF/THEN/ELSE/ENDIF
1078 %                for ternary expressions
1079 % #2 (optional)  -- keys: alias, aliasleft, aliasright
1080 %                alias      -- replaces the whole display row
1081 %                aliasleft  -- replaces the left-hand side
1082 %                (symbol)
1083 %                aliasright -- replaces the right-hand side
1084 %                (calculation)
1085 %                Use \cdots for omitted parts.
1086 % #3             -- name (string) of the left-hand variable
1087 % #4             -- calculation (with \modXxx macros and \fpeval
1088 %                syntax)
1089 %
1090 % With alias or aliasright the ternary logic is skipped (display

```

```

1091 % shows the alias content verbatim, not IF/THEN/ELSE). Execution
1092 % always uses the calculation from #4.
1093 %
1094 % The calculation accepts all \fpeval operators:
1095 % +, -, *, /, ^, sign(), abs(), sin(), cos(), sqrt(), round()
1096 % Ternary: cond ? expr_true : expr_false
1097 % Logical: && (AND), || (OR), comparisons (<, >, <=, >=)
1098 %
1099 % The rule is stored for both display (\textmodel) and execution
1100 % (\computemodel).
1101 %
1102 % Examples:
1103 % \mrule{modFres}{\modM * \modA}
1104 % \mrule{modFw}{sign(\modV) * \modK * \modV^2}
1105 % \mrule{modA}{(\modT < \modTq) || (\modT > \modTdq) ? \modA + \modDa : \modA - \modDa}
1106 % \mrule[aliasright=\cdots]{modT}{\modT + \modDt}
1107 % \mrule[aliasleft=a_x]{modAx}{\modFgx / \modM}
1108 % \mrule[alias=\text{(hidden)}]{modAy}{\modFgy / \modM}
1109
1110 \NewDocumentCommand{\mrule}{s O{} m m }{%
1111 \typeout{MRULE~start:~#3}
1112 \bool_set_false:N \l__numodel_is_ternary_bool
1113 % Parse keys (prefix, alias, aliasleft, aliasright; gridx/gridy ignored)
1114 \tl_clear:N \l__numodel_alias_tl
1115 \tl_clear:N \l__numodel_aliasleft_tl
1116 \tl_clear:N \l__numodel_aliasright_tl
1117 \tl_clear:N \l__numodel_cmd_prefix_tl
1118 \keys_set:nn { numodel / mvar } {#2}
1119 \__numodel_resolve_eff_prefix:
1120 \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1121 { \__numodel_mrule_body:nnn {#1}{#3}{#4} }%
1122 \typeout{MRULE~done:~#3}
1123 }
1124
1125 \cs_new_protected:Npn \__numodel_mrule_body:nnn #1 #2 #3
1126 {
1127 \int_gincr:N \g_mrule_counter_int
1128 \tl_set:Ne \l__numodel_fullname_tl { \g_numodel_current_prefix_tl #2 }
1129 % Store execution data (target = full name)
1130 \seq_gput_right:Nx \g_mrule_calc_seq
1131 { { \l__numodel_fullname_tl } { \exp_not:n {#3} } }
1132 % Left-hand side: aliasleft or default display name
1133 \tl_if_blank:VTF \l__numodel_aliasleft_tl
1134 { \tl_set:Ne \l__numodel_lhs_tl { \use:c { \l__numodel_fullname_tl text } } }
1135 { \tl_set_eq:NN \l__numodel_lhs_tl \l__numodel_aliasleft_tl }
1136 % Generate display
1137 \tl_if_blank:VTF \l__numodel_alias_tl
1138 {
1139 \tl_if_blank:VTF \l__numodel_aliasright_tl
1140 {
1141 % Automatic display generation (ternary or normal)
1142 \__numodel_parse_ternary:nN {#3} \l__numodel_display_tl
1143 \bool_if:NTF \l__numodel_is_ternary_bool
1144 {
1145 \IfBooleanTF {#1}
1146 {
1147 % Starred ternary -> multiline IF/THEN/ELSE/ENDIF
1148 \tl_set:Ne \l__numodel_display_tl
1149 {
1150 \__numodel_kwt:n {if}
1151 \exp_not:V \l__numodel_cond_tl
1152 \__numodel_kwt:n {then_n}
1153 }
1154 \seq_gput_right:NV \g_mrule_seq \l__numodel_display_tl
1155 \seq_gput_right:Nn \g_mrule_type_seq { rule }
1156 \tl_set:Ne \l__numodel_display_tl

```

```

1157         {
1158             \exp_not:n { \quad }
1159             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, , }
1160             \exp_not:V \l__numodel_true_tl
1161         }
1162         \seq_gput_right:NV \g_mrule_seq \l__numodel_display_tl
1163         \seq_gput_right:Nn \g_mrule_type_seq { cont }
1164         \seq_gput_right:Ne \g_mrule_seq { \__numodel_kwt:n {else_n1} }
1165         \seq_gput_right:Nn \g_mrule_type_seq { cont }
1166         \tl_set:Ne \l__numodel_display_tl
1167         {
1168             \exp_not:n { \quad }
1169             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, , }
1170             \exp_not:V \l__numodel_false_tl
1171         }
1172         \seq_gput_right:NV \g_mrule_seq \l__numodel_display_tl
1173         \seq_gput_right:Nn \g_mrule_type_seq { cont }
1174         \seq_gput_right:Ne \g_mrule_seq { \__numodel_kwt:n {endif_n1} }
1175         \seq_gput_right:Nn \g_mrule_type_seq { cont }
1176     }
1177     {
1178         % Unstarred ternary -> single-line
1179         \tl_set:Ne \l__numodel_display_tl
1180         {
1181             \__numodel_kwt:n {if}
1182             \exp_not:V \l__numodel_cond_tl
1183             \__numodel_kwt:n {then}
1184             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, , }
1185             \exp_not:V \l__numodel_true_tl
1186             \__numodel_kwt:n {else}
1187             \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, , }
1188             \exp_not:V \l__numodel_false_tl
1189             \__numodel_kwt:n {endif}
1190         }
1191     }
1192 }
1193 {
1194     % Ordinary assignment: lhs = rhs
1195     \tl_set:Nn \l__numodel_rhs_tl {#3}
1196     \__numodel_vars_to_display:N \l__numodel_rhs_tl
1197     \typeout{MRULE~rhs~after:~\tl_to_str:N \l__numodel_rhs_tl}
1198     \tl_set:Ne \l__numodel_display_tl
1199     {
1200         \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, , }
1201         \exp_not:V \l__numodel_rhs_tl
1202     }
1203     \typeout{MRULE~display:~\tl_to_str:N \l__numodel_display_tl}
1204 }
1205 }
1206 {
1207     % aliasright: lhs = aliasright (no ternary processing)
1208     \tl_set:Ne \l__numodel_display_tl
1209     {
1210         \exp_not:V \l__numodel_lhs_tl \exp_not:n { \,=\, , }
1211         \exp_not:V \l__numodel_aliasright_tl
1212     }
1213 }
1214 }
1215 {
1216     % alias: replace the whole row
1217     \tl_set:Ne \l__numodel_display_tl { \exp_not:V \l__numodel_alias_tl }
1218 }
1219 % For starred ternary everything is already pushed above
1220 \bool_if:nF { \l__numodel_is_ternary_bool && #1 }
1221 {
1222     \seq_gput_right:NV \g_mrule_seq \l__numodel_display_tl

```

```

1223     \seq_gput_right:Nn \g_mrule_type_seq { rule }
1224   }
1225   % Pillar A - Lua-side rule registration. Detokenize the raw
1226   % expression so Lua sees the macro names, not the evaluated
1227   % fp values. Lua handles both the dependency extraction
1228   % (build_deps) and the flow detection (classify_flows) at
1229   % \graphicmodel time.
1230   \tl_set:Nx \l__numodel_scratch_tl { \detokenize {#3} }
1231   \directlua{ numodel.add_rule(
1232     "g_numodel_current_prefix_tl",
1233     "\l__numodel_fullname_tl",
1234     "\luaescapestring{\l__numodel_scratch_tl}",
1235     "\bool_if:NTF \l__numodel_is_ternary_bool { ternary } { calc }") }
1236   }
1237
1238 % =====
1239 % \mruletext{free text}
1240 % =====
1241 % Adds a free-text row in the model table. Useful for structure
1242 % that does not fit as \mrule, e.g.:
1243 % \mruletext{\text{IF } t < T/4 \text{ THEN}}
1244 % \mruletext{\quad a = a + da}
1245 % \mruletext{\text{ENDIF}}
1246 %
1247 % NOT executed by \computemodel (display only).
1248
1249 \NewDocumentCommand{\mruletext}{O{} m }{%
1250   \tl_clear:N \l__numodel_cmd_prefix_tl
1251   \keys_set:nn { numodel / cmd } {#1}
1252   \__numodel_resolve_eff_prefix:
1253   \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1254   {
1255     \int_gincr:N \g_mrule_counter_int
1256     \seq_gput_right:Nn \g_mrule_seq {#2}
1257     \seq_gput_right:Nn \g_mrule_type_seq { rule }
1258   }
1259 }
1260
1261 % =====
1262 % \mstop[keys]{condition}
1263 % \mstop*[keys]{condition} (star reserved; currently identical)
1264 % =====
1265 % Declare the model's stop condition.
1266 % Display: "IF condition THEN STOP ENDIF"
1267 % Execution: the model stops when the condition is true (evaluates to 1).
1268 %
1269 % The condition uses \fpeval syntax with \mvar variables:
1270 % \mstop{\modT >= \modTmax}
1271 % \mstop{\modV <= 0}
1272 %
1273 % Supported keys:
1274 % alias={...} -- replaces the whole display row (execution
1275 % remains intact)
1276 % aliasleft/aliasright are not (yet) supported for \mstop.
1277
1278 \tl_new:N \l__numodel_stop_tl
1279 \NewDocumentCommand{\mstop}{s O{} m }{%
1280   \typeout{MSTOP~start}
1281   % Parse keys (only alias is honoured; prefix via key resolver)
1282   \tl_clear:N \l__numodel_alias_tl
1283   \tl_clear:N \l__numodel_aliasleft_tl
1284   \tl_clear:N \l__numodel_aliasright_tl
1285   \tl_clear:N \l__numodel_cmd_prefix_tl
1286   \keys_set:nn { numodel / mvar } {#2}
1287   \__numodel_resolve_eff_prefix:
1288   \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl

```

```

1289 { \_numodel_mstop_body:n {#3} }%
1290 \typeout{MSTOP~done}
1291 }
1292
1293 \cs_new_protected:Npn \_numodel_mstop_body:n #1
1294 {
1295   \int_gincr:N \g_mruler_counter_int
1296   % Store expression for execution (always the actual condition)
1297   \tl_gset:Nn \g_numodel_stop_expr_tl {#1}
1298   % Generate display
1299   \tl_if_blank:VTF \l__numodel_alias_tl
1300   {
1301     \tl_set:Nn \l__numodel_stop_tl {#1}
1302     \_numodel_vars_to_display:N \l__numodel_stop_tl
1303     \tl_set:Ne \l__numodel_display_tl
1304     {
1305       \_numodel_kwt:n {if}
1306       \exp_not:V \l__numodel_stop_tl
1307       \_numodel_kwt:n {then_n}
1308       \_numodel_kwt:n {stop}
1309       \_numodel_kwt:n {endif_n}
1310     }
1311   }
1312   {
1313     \tl_set:Ne \l__numodel_display_tl { \exp_not:V \l__numodel_alias_tl }
1314   }
1315   \seq_gput_right:N \g_mruler_seq \l__numodel_display_tl
1316   \seq_gput_right:Nn \g_mruler_type_seq { rule }
1317 }
1318
1319 % =====
1320 % \textmodel
1321 % =====
1322 % Builds a tabular with numbered model rules on the left and initial
1323 % values on the right. Can be placed anywhere, e.g. in a subfigure
1324 % next to a graphic model.
1325
1326 \tl_new:N \g__numodel_table_tl
1327 \int_new:N \l__numodel_row_int
1328 \int_new:N \l__numodel_dispnum_int
1329 \int_new:N \l__numodel_startrow_int
1330 \int_new:N \l__numodel_numrules_int
1331 \int_new:N \l__numodel_numstarts_int
1332
1333 % Emit one initial-value cell with alias-key support.
1334 % - \<name>alias not empty -> replaces whole cell (inside $...$)
1335 % - \<name>aliasleft not empty -> replaces left symbol, else
1336 % \<name>text
1337 % - \<name>aliasright not empty -> replaces right value, else
1338 % \<name>qty (units=true) or
1339 % \<name>num (units=false)
1340 \cs_new_protected:Npn \_numodel_emit_startcell:n #1
1341 {
1342   \tl_if_blank:eTF { \use:c { #1 alias } }
1343   {
1344     \tl_gput_right:Nn \g__numodel_table_tl { $ }
1345     \tl_if_blank:eTF { \use:c { #1 aliasleft } }
1346     {
1347       \tl_gput_right:Ne \g__numodel_table_tl
1348       { \exp_not:e { \use:c { #1 text } } }
1349     }
1350     {
1351       \tl_gput_right:Ne \g__numodel_table_tl
1352       { \exp_not:e { \use:c { #1 aliasleft } } }
1353     }
1354     \tl_gput_right:Nn \g__numodel_table_tl { = }

```



```

1355 \tl_if_blank:eeTF { \use:c { #1 aliasright } }
1356 {
1357     % Emit \<name>qty or \<name>num as a token name without
1358     % expanding it; siunitx macros must only expand at
1359     % \tl_use:N time, in the correct tabular context.
1360     \bool_if:NTF \g__numodel_units_bool
1361     {
1362         \tl_gput_right:Nx \g__numodel_table_tl
1363         { \exp_not:c { #1 qty } }
1364     }
1365     {
1366         \tl_gput_right:Nx \g__numodel_table_tl
1367         { \exp_not:c { #1 num } }
1368     }
1369 }
1370 {
1371     \tl_gput_right:Ne \g__numodel_table_tl
1372     { \exp_not:e { \use:c { #1 aliasright } } }
1373 }
1374 \tl_gput_right:Nn \g__numodel_table_tl { $ }
1375 }
1376 {
1377     \tl_gput_right:Nn \g__numodel_table_tl { $ }
1378     \tl_gput_right:Ne \g__numodel_table_tl
1379     { \exp_not:e { \use:c { #1 alias } } }
1380     \tl_gput_right:Nn \g__numodel_table_tl { $ }
1381 }
1382 }
1383
1384 \cs_new_protected:Npn \__numodel_build_table:
1385 {
1386     \typeout{BUILD_TABLE~rules:~\seq_use:Nn \g_mrule_seq {||}}
1387     \typeout{BUILD_TABLE~types:~\seq_use:Nn \g_mrule_type_seq {||}}
1388     \int_set:Nn \l__numodel_numrules_int { \seq_count:N \g_mrule_seq }
1389     \int_set:Nn \l__numodel_numstarts_int { \seq_count:N \g_mvar_start_seq }
1390     \typeout{BUILD_TABLE~numrules:~\int_use:N \l__numodel_numrules_int}
1391     \typeout{BUILD_TABLE~numstarts:~\int_use:N \l__numodel_numstarts_int}
1392     \int_zero:N \l__numodel_row_int
1393     \int_zero:N \l__numodel_dispnum_int
1394     \int_zero:N \l__numodel_startrow_int
1395     \tl_gset:Nn \g__numodel_table_tl { \begin }
1396     \tl_gput_right:Ne \g__numodel_table_tl
1397     { { \tl_use:N \g__numodel_tblrenv_tl } }
1398     \tl_gput_right:Nn \g__numodel_table_tl
1399     { [theme={numodel}]{colspec={r|l|l}, rowhead=1} & \textbf }
1400     \tl_gput_right:Ne \g__numodel_table_tl
1401     { { \__numodel_kw:n {th_model} } }
1402     \tl_gput_right:Nn \g__numodel_table_tl
1403     { & \textbf }
1404     \tl_gput_right:Ne \g__numodel_table_tl
1405     { { \__numodel_kw:n {th_initvals} } }
1406     \tl_gput_right:Nn \g__numodel_table_tl
1407     { \\ \hline }
1408     \typeout{BUILD_TABLE~before-step~table:~\tl_to_str:N \g__numodel_table_tl}
1409     \int_step_inline:nn { \l__numodel_numrules_int }
1410     {
1411         \typeout{BUILD_TABLE~iter~row:~\int_use:N \l__numodel_row_int}
1412         \int_incr:N \l__numodel_row_int
1413         \int_incr:N \l__numodel_startrow_int
1414         \typeout{BUILD_TABLE~row:~\int_use:N \l__numodel_row_int~type:~\seq_item:Nn \g_mrule_type_seq
1415         % Check type: rule -> show number, cont -> blank
1416         \str_if_eq:eeTF
1417         { \seq_item:Nn \g_mrule_type_seq { \l__numodel_row_int } }
1418         { cont }
1419         {
1420             % Continuation rows: no number

```

```

1421         \tl_gput_right:Ne \g__numodel_table_tl
1422         {
1423             \exp_not:n { \rule{0pt}{2.6ex} }
1424             \exp_not:n { & $ }
1425             \exp_not:e { \seq_item:Nn \g_mrule_seq { \l__numodel_row_int } }
1426             \exp_not:n { $ & }
1427         }
1428     }
1429     {
1430         % Numbered row
1431         \int_incr:N \l__numodel_dispnum_int
1432         \typeout{BUILD_TABLE~emit~rule~num::~~\int_use:N \l__numodel_dispnum_int~content::~~\seq_item:Nn \g_mrule_seq { \l__numodel_row_int }}
1433         \tl_gput_right:Ne \g__numodel_table_tl
1434         {
1435             \exp_not:n { \rule{0pt}{2.6ex} }
1436             \int_use:N \l__numodel_dispnum_int
1437             \exp_not:n { & $ }
1438             \exp_not:e { \seq_item:Nn \g_mrule_seq { \l__numodel_row_int } }
1439             \exp_not:n { $ & }
1440         }
1441         \typeout{BUILD_TABLE~after~emit~table~tail::~~\tl_tail:N \g__numodel_table_tl}
1442     }
1443     % Initial-value column (independent of rule/cont)
1444     \int_compare:nNnTF { \l__numodel_startrow_int } > { \l__numodel_numstarts_int }
1445     {
1446         \tl_gput_right:Nn \g__numodel_table_tl { \\ }
1447     }
1448     {
1449         \exp_args:Ne \__numodel_emit_startcell:n
1450         { \seq_item:Nn \g_mvar_start_seq { \l__numodel_startrow_int } }
1451         \tl_gput_right:Nn \g__numodel_table_tl { \\ }
1452     }
1453 }
1454 \int_while_do:nNnn { \l__numodel_startrow_int } < { \l__numodel_numstarts_int }
1455 {
1456     \int_incr:N \l__numodel_startrow_int
1457     \tl_gput_right:Nn \g__numodel_table_tl { & & }
1458     \exp_args:Ne \__numodel_emit_startcell:n
1459     { \seq_item:Nn \g_mvar_start_seq { \l__numodel_startrow_int } }
1460     \tl_gput_right:Nn \g__numodel_table_tl { \\ }
1461 }
1462 \tl_gput_right:Nn \g__numodel_table_tl { \end }
1463 \tl_gput_right:Ne \g__numodel_table_tl
1464 { { \tl_use:N \g__numodel_tblrenv_tl } }
1465 }
1466
1467 \NewDocumentCommand{\textmodel}{O{}} {%
1468     \typeout{TEXTMODEL~start}
1469     \tl_clear:N \l__numodel_cmd_prefix_tl
1470     \tl_clear:N \l__numodel_cmd_units_tl
1471     \tl_clear:N \l__numodel_cmd_tblrenv_tl
1472     \keys_set:nn { numodel / cmd } {#1}
1473     \__numodel_resolve_eff_prefix:
1474     % Temporary units override: save the global value, apply the key
1475     % value before build, restore afterwards. This way
1476     % \textmodel[units=false] flips one render without touching the
1477     % global \numodelsetup state.
1478     \bool_set_eq:NN \l__numodel_saved_units_bool \g__numodel_units_bool
1479     \tl_if_empty:NF \l__numodel_cmd_units_tl
1480     {
1481         \str_if_eq:VnTF \l__numodel_cmd_units_tl { true }
1482         { \bool_gset_true:N \g__numodel_units_bool }
1483         { \bool_gset_false:N \g__numodel_units_bool }
1484     }
1485     % Same dance for tblrenv: save, apply per-call override, restore.
1486     \tl_set_eq:NN \l__numodel_saved_tblrenv_tl \g__numodel_tblrenv_tl

```

```

1487 \tl_if_empty:NF \l__numodel_cmd_tblrenv_tl
1488 { \tl_gset_eq:NN \g__numodel_tblrenv_tl \l__numodel_cmd_tblrenv_tl }
1489 \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1490 {
1491   \group_begin:
1492     \__numodel_apply_dsep:
1493     \__numodel_build_table:
1494     \typeout{TEXTMODEL~table:~\tl_to_str:N \g__numodel_table_tl}
1495     \tl_use:N \g__numodel_table_tl
1496   \group_end:
1497 }%
1498 \bool_gset_eq:NN \g__numodel_units_bool \l__numodel_saved_units_bool
1499 \tl_gset_eq:NN \g__numodel_tblrenv_tl \l__numodel_saved_tblrenv_tl
1500 \typeout{TEXTMODEL~done}
1501 }
1502 \bool_new:N \l__numodel_saved_units_bool
1503 \tl_new:N \l__numodel_saved_tblrenv_tl
1504
1505 % =====
1506 % \computemodel
1507 % =====
1508 % Run the model with the Euler method:
1509 %   1. Record all variable values in Lua
1510 %   2. Check stop condition (\mstop)
1511 %   3. Execute every \mrule rule (in declaration order)
1512 %   4. Repeat until stop or safety limit (default 20000 iterations)
1513 %
1514 % Requires: at least one \mstop and at least one \mrule.
1515 % Performance: ~440 steps in ~4s, ~20000 steps in ~60s.
1516
1517 \cs_new_protected:Npn \__numodel_exec_rule:nn #1 #2
1518 {
1519   \cs_gset:cpe {#1} { \fp_eval:n {#2} }
1520 }
1521
1522 % Record all current variable values in Lua (O(1) per variable).
1523 \cs_new_protected:Npn \__numodel_lua_record_all:
1524 {
1525   \seq_map_inline:Nn \g_mvar_names_seq
1526   {
1527     \directlua{ numodel.record(
1528       "\g_numodel_current_prefix_tl", "##1", \use:c{##1}) }
1529   }
1530   \directlua{ numodel.end_step("\g_numodel_current_prefix_tl") }
1531 }
1532
1533 % Set min/max TeX macros from Lua data after the simulation finishes.
1534 \cs_new_protected:Npn \__numodel_set_minmax_from_lua:
1535 {
1536   \seq_map_inline:Nn \g_mvar_names_seq
1537   {
1538     \cs_gset:cpe { ##1 min }
1539     { \directlua{ numodel.get_min("\g_numodel_current_prefix_tl", "##1") } }
1540     \cs_gset:cpe { ##1 max }
1541     { \directlua{ numodel.get_max("\g_numodel_current_prefix_tl", "##1") } }
1542   }
1543 }
1544
1545
1546 \NewDocumentCommand{\computemodel}{ 0 } {} {%
1547   \typeout{COMPUTE~start}
1548   \tl_clear:N \l__numodel_cmd_prefix_tl
1549   \keys_set:nn { numodel / cmd } {#1}
1550   \__numodel_resolve_eff_prefix:
1551   \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
1552   { \__numodel_compute_body: }%

```

```

1553 \typeout{COMPUTE~done}
1554 }
1555
1556 \cs_new_protected:Npn \__numodel_compute_body:
1557 {
1558   \tl_if_empty:NT \g_numodel_stop_expr_tl
1559   { \msg_error:nn { numodel } { no-stop } }
1560   \int_gzero:N \g_numodel_steps_int
1561   % Initialise Lua storage for this prefix
1562   \directlua{ numodel.init("\g_numodel_current_prefix_tl") }
1563   % Main loop
1564   \bool_gset_false:N \g_tmpa_bool
1565   \bool_do_until:Nn \g_tmpa_bool
1566   {
1567     \__numodel_lua_record_all:
1568     \int_gincr:N \g_numodel_steps_int
1569     \int_compare:nNnT
1570       { \fp_eval:n { \g_numodel_stop_expr_tl } } = { 1 }
1571     { \bool_gset_true:N \g_tmpa_bool }
1572     \bool_if:NF \g_tmpa_bool
1573     {
1574       \seq_map_inline:Nn \g_mrule_calc_seq
1575       { \__numodel_exec_rule:nn ##1 }
1576     }
1577     \int_compare:nNnT
1578       { \g_numodel_steps_int } > { \g_numodel_maxiter_int }
1579     {
1580       \bool_gset_true:N \g_tmpa_bool
1581       \msg_warning:nn { numodel } { maxiter }
1582     }
1583   }
1584   \__numodel_set_minmax_from_lua:
1585 }
1586
1587 % On-demand coordinates from Lua (after \computemodel). Arguments
1588 % are SHORT names; the current prefix is prepended automatically.
1589 % See \mcoordsp for the form with an explicit prefix.
1590 %
1591 % Fully expandable (a plain \def around \directlua, no xparse
1592 % wrapper). Works directly inside \addplot coordinates
1593 % {\mcoords{T}{V}} and pgfplots' math-expression parser, without
1594 % pre-expansion via \edef.
1595 \cs_new:Npn \mcoords #1#2
1596 {
1597   \directlua{ numodel.get_coords(
1598     "\g_numodel_current_prefix_tl",
1599     "\g_numodel_current_prefix_tl" .. "#1",
1600     "\g_numodel_current_prefix_tl" .. "#2") }
1601 }
1602
1603 % Variant with explicit prefix as first argument (instead of an
1604 % optional argument, so the macro stays fully expandable).
1605 \cs_new:Npn \mcoordsp #1#2#3
1606 {
1607   \directlua{ numodel.get_coords(
1608     "#1", "#1" .. "#2", "#1" .. "#3") }
1609 }
1610
1611 % Value of variable #1 at step #2 (0-based). The variable name is
1612 % SHORT (current prefix is prepended automatically). See \mstepp for
1613 % the form with an explicit prefix.
1614 %
1615 % Example -- tangent line at step 0:
1616 % \addplot[domain=0:\modTmax]
1617 %   {\mstep{V}{0} + \mstep{A}{1} * (x - \mstep{T}{0})};
1618 \cs_new:Npn \mstep #1#2

```

```

1619 {
1620   \directlua{ numodel.get_step(
1621     "\g_numodel_current_prefix_tl",
1622     "\g_numodel_current_prefix_tl" .. "#1", #2) }
1623 }
1624
1625 \cs_new:Npn \mstepp #1#2#3
1626 {
1627   \directlua{ numodel.get_step("#1", "#1" .. "#2", #3) }
1628 }
1629
1630 % =====
1631 % \modelreset -- REMOVED
1632 % =====
1633 % \modelreset no longer exists. Use \newmodelprefix{<prefix>} to set
1634 % up a new model and \switchmodelprefix{<prefix>} to switch between
1635 % existing ones. Namespaces are additive -- variables from earlier
1636 % models remain available.
1637
1638 % =====
1639 % \graphicmodel -- Forrester diagram from \mvar/\mrule data
1640 % =====
1641 % Builds a tikzpicture with:
1642 %   - stock, valve, aux and const nodes based on type and gridx/gridy
1643 %   - flow arrows from valve to stock (via \flowarrow)
1644 %   - causal arrows from the dependency graph
1645 %
1646 % Positioning: automatic (auto-layout) or manual via
1647 %   \mvar[gridx=N, gridy=N]{...}. Mixed mode is supported.
1648 % Auto-layout places stocks+valves at gridy=0, aux at gridy=1, and
1649 % constants at gridy=2. Variables of type system are skipped.
1650 % Auxiliary variables that act as inflow are placed as a valve
1651 % (not as aux).
1652
1653 \tl_new:N \g_numodel_graphic_tl
1654 % Lua-populated cache (filled by numodel.tex_writeback at the start of
1655 % each \graphicmodel call, consumed by the emit helpers below).
1656 \prop_new:N \l_numodel_valve_for_prop % aux/const -> stock (inflow)
1657 % Secondary stocks for a shared inflow valve. Value = comma-list of
1658 % stock names; the renderer iterates it to draw a curved branch from
1659 % the valve, sweeping over the primary stock into each extra target.
1660 \prop_new:N \l_numodel_valve_extras_prop
1661 \prop_new:N \l_numodel_outvalve_for_prop % aux/const -> stock (outflow)
1662 % Mirror of valve_extras_prop for shared outflow valves: a curved
1663 % branch from each extra source stock arcs over the intermediate
1664 % stocks into the shared valve's open end.
1665 \prop_new:N \l_numodel_outvalve_extras_prop
1666 \prop_new:N \l_numodel_between_valve_prop % aux/const -> source_stock (between-flow)
1667 \prop_new:N \l_numodel_between_target_prop % aux/const -> target_stock (between-flow)
1668 \prop_new:N \l_numodel_stock_valve_prop % stock -> source stock (stock-as-flow)
1669 \prop_new:N \l_numodel_stock_phantom_valve_prop % stock -> source stock (phantom flow)
1670 % For diagram-style=forrester|edu: separate valve gridx position next
1671 % to the stock; the variable's own gridx/gridy keeps the natural
1672 % position at gridy=1 (aux) or gridy=2 (constant). vpos_y is always 0
1673 % so it lives implicitly in the emit helpers.
1674 \prop_new:N \l_numodel_vpos_x_prop % varname -> valve gridx
1675 \prop_new:N \l_numodel_vpos_y_prop % varname -> valve gridy (used when gridmaxx wrap shifted
1676 \tl_new:N \l_numodel_flows_tl % deferred flow arrows
1677 \tl_new:N \l_numodel_valves_tl % deferred valve nodes (drawn on top)
1678 \tl_new:N \l_numodel_late_causals_tl % causals pointing at valves
1679
1680 % --- Resolved render settings (set per-\graphicmodel call) ---
1681 % These hold the effective values (after resolving diagram-style
1682 % defaults and explicit global/local overrides) used by the emit
1683 % helpers and by the public \flowarrow / \flowoutarrow macros.
1684 \tl_new:N \l_numodel_flowarrow_eff_tl % "hollow" or "filled"

```

```

1685 \tl_new:N \l__numodel_valve_eff_tl          % "valve" | "circle" | "edu"
1686 \tl_new:N \l__numodel_flowcloud_global_tl    % "true" or "false" (global default)
1687
1688 % Resolve flowarrow-style: explicit global key wins; otherwise
1689 % diagram-style picks the default (forrester -> hollow,
1690 % tight|edu -> filled).
1691 \cs_new_protected:Npn \__numodel_resolve_flowarrow:
1692 {
1693   \tl_if_empty:NTF \g__numodel_flowarrow_style_tl
1694   {
1695     \str_if_eq:VnTF \g__numodel_diagram_style_tl { forrester }
1696     { \tl_set:Nn \l__numodel_flowarrow_eff_tl { hollow } }
1697     { \tl_set:Nn \l__numodel_flowarrow_eff_tl { filled } }
1698   }
1699   { \tl_set_eq:NN \l__numodel_flowarrow_eff_tl
1700     \g__numodel_flowarrow_style_tl }
1701 }
1702
1703 % Resolve valve-style: explicit global key wins; otherwise
1704 % diagram-style picks the default (forrester -> valve,
1705 % tight|edu -> edu).
1706 \cs_new_protected:Npn \__numodel_resolve_valve:
1707 {
1708   \tl_if_empty:NTF \g__numodel_valve_style_tl
1709   {
1710     \str_if_eq:VnTF \g__numodel_diagram_style_tl { forrester }
1711     { \tl_set:Nn \l__numodel_valve_eff_tl { valve } }
1712     { \tl_set:Nn \l__numodel_valve_eff_tl { edu } }
1713   }
1714   { \tl_set_eq:NN \l__numodel_valve_eff_tl
1715     \g__numodel_valve_style_tl }
1716 }
1717
1718 % Resolve the global flowarrow-cloud-tip default (no per-stock
1719 % override yet; that's added on top in \__numodel_eff_flowcloud:n).
1720 \cs_new_protected:Npn \__numodel_resolve_flowcloud:
1721 {
1722   \tl_if_empty:NTF \g__numodel_flowcloud_tl
1723   {
1724     \str_if_eq:VnTF \g__numodel_diagram_style_tl { forrester }
1725     { \tl_set:Nn \l__numodel_flowcloud_global_tl { true } }
1726     { \tl_set:Nn \l__numodel_flowcloud_global_tl { false } }
1727   }
1728   { \tl_set_eq:NN \l__numodel_flowcloud_global_tl
1729     \g__numodel_flowcloud_tl }
1730 }
1731
1732 % Per-stock flowcloud lookup. #1 = stock varname. Sets
1733 % \l__numodel_flowcloud_eff_tl to "true" or "false". Order: per-mvar
1734 % override on the stock (via [flowarrow-cloud-tip=...]) wins over the
1735 % global default.
1736 \tl_new:N \l__numodel_flowcloud_eff_tl
1737 \cs_new_protected:Npn \__numodel_eff_flowcloud:n #1
1738 {
1739   \tl_set:Nn \l__numodel_flowcloud_eff_tl { \use:c { #1 flowcloud } }
1740   \tl_if_empty:NT \l__numodel_flowcloud_eff_tl
1741   { \tl_set_eq:NN \l__numodel_flowcloud_eff_tl
1742     \l__numodel_flowcloud_global_tl }
1743 }
1744
1745 % Sets the public \nmflowbody and \nmflowtip macros that the
1746 % \flowarrow / \flowoutarrow / \flowbetweenarrow macros expand to.
1747 % Reads the resolved flowarrow-style.
1748 \cs_new_protected:Npn \__numodel_apply_flowarrow_style:
1749 {
1750   \str_if_eq:VnTF \l__numodel_flowarrow_eff_tl { hollow }

```

```

1751     {
1752         \cs_gset:Npn \nmflowbody { flowpipe-hollow }
1753         \cs_gset:Npn \nmflowtip { flowpipe-hollow-tip }
1754         \cs_gset:Npn \nmflowtipcloudin { flowpipe-hollow-cloudin }
1755         \cs_gset:Npn \nmflowtipcloudout { flowpipe-hollow-cloudout }
1756     }
1757     {
1758         \cs_gset:Npn \nmflowbody { flowpipe-filled }
1759         \cs_gset:Npn \nmflowtip { flowpipe-filled-tip }
1760         \cs_gset:Npn \nmflowtipcloudin { flowpipe-filled-cloudin }
1761         \cs_gset:Npn \nmflowtipcloudout { flowpipe-filled-cloudout }
1762     }
1763 }
1764
1765 % Sets the tikz node-style name for the valve and the boolean that
1766 % controls whether the valve carries the variable's display label.
1767 % valve-style=valve -> [valve-forrester], no label
1768 % valve-style=circle -> [valve-circle], no label
1769 % valve-style=edu -> [valve-edu], with label
1770 \tl_new:N \l__numodel_valve_node_tl
1771 \bool_new:N \l__numodel_valve_label_bool
1772 \cs_new_protected:Npn \__numodel_apply_valve_style:
1773 {
1774     \str_case:VnF \l__numodel_valve_eff_tl
1775     {
1776         { valve }
1777         {
1778             \tl_set:Nn \l__numodel_valve_node_tl { valve-forrester }
1779             \bool_set_false:N \l__numodel_valve_label_bool
1780         }
1781         { circle }
1782         {
1783             \tl_set:Nn \l__numodel_valve_node_tl { valve-circle }
1784             \bool_set_false:N \l__numodel_valve_label_bool
1785         }
1786         { edu }
1787         {
1788             \tl_set:Nn \l__numodel_valve_node_tl { valve-edu }
1789             \bool_set_true:N \l__numodel_valve_label_bool
1790         }
1791     }
1792     {
1793         \tl_set:Nn \l__numodel_valve_node_tl { valve-edu }
1794         \bool_set_true:N \l__numodel_valve_label_bool
1795     }
1796 }
1797
1798 % --- Diagram-style mode flag ---
1799 % True at diagram-style=forrester|edu; consulted by \__numodel_place_node
1800 % to decide between single-emit and natural+phantom-valve double-emit.
1801 % Set in \__numodel_build_graphic from \g__numodel_diagram_style_tl.
1802 \bool_new:N \l__numodel_keep_natural_bool
1803
1804
1805 \cs_new_protected:Npn \__numodel_build_graphic:
1806 {
1807     \typeout{BUILD:~step-0~reset~auto~positions}
1808     % --- Reset gridx/gridy to the original user input ---
1809     % Auto-layout writes positions to var.gridx/gridy. On a second
1810     % \graphicmodel call those would, without reset, be treated as
1811     % "manually placed". The initial values are saved in
1812     % gridxinit/gridyinit by \mvar -- copy them back.
1813     \seq_map_inline:Nn \g_mvar_names_seq
1814     {
1815         \cs_gset:cpe { ##1 gridx } { \use:c { ##1 gridxinit } }
1816         \cs_gset:cpe { ##1 gridy } { \use:c { ##1 gridyinit } }

```

```

1817     }
1818     % Diagram-style mode: forrester|edu keep aux/const at their natural
1819     % gridy with a phantom valve next to the stock; tight collapses the
1820     % aux/const onto the valve position. Consumed by \_numodel_place_node
1821     % to dispatch between single-emit and natural+phantom emit.
1822     \bool_set_false:N \l__numodel_keep_natural_bool
1823     \str_if_eq:VnT \g__numodel_diagram_style_tl { forrester }
1824         { \bool_set_true:N \l__numodel_keep_natural_bool }
1825     \str_if_eq:VnT \g__numodel_diagram_style_tl { edu }
1826         { \bool_set_true:N \l__numodel_keep_natural_bool }
1827     \typeout{BUILD:~step~1~lua~layout~writeback}
1828     % Pillar A - Lua is the single source of truth for flow detection
1829     % and auto-layout. The writeback clears and fills \<var>gridx/gridy
1830     % and the flow-props that downstream emitters (place_node,
1831     % flow-builders, emit_natural_and_phantom, emit_stock_valve) read.
1832     \_numodel_lua_layout_writeback:
1833     \typeout{BUILD:~step~2~tikzpicture}
1834     % --- Build tikzpicture ---
1835     % Render order matters: each segment overlays the previous one,
1836     % so we draw flows first (one continuous arrow per flow), then
1837     % the valve nodes on top (with white fill so they cover the
1838     % section of the arrow that lies underneath), and finally the
1839     % causal arrows that point at those valves.
1840     \tl_gclear:N \g__numodel_graphic_tl
1841     \tl_clear:N \l__numodel_flows_tl
1842     \tl_clear:N \l__numodel_valves_tl
1843     \tl_clear:N \l__numodel_late_causals_tl
1844     \tl_gput_right:Nn \g__numodel_graphic_tl
1845         { \begin{tikzpicture}[gridscale] }
1846     % Nodes (stocks, aux, const, clouds; flows -> flows_tl,
1847     % valves -> valves_tl)
1848     \seq_map_inline:Nn \g_mvar_names_seq
1849         { \typeout{NODE:~##1} \_numodel_place_node:n {##1} }
1850     \typeout{BUILD:~step~5~stock-valve~nodes}
1851     % Stock-as-flow valve nodes
1852     \prop_map_inline:Nn \l__numodel_stock_valve_prop
1853         {
1854             % ##1 = stock (e.g. modH), ##2 = source stock (e.g. modV)
1855             % Skip entries that are not real stock-valves (e.g. __svgx keys)
1856             \tl_if_in:nnF {##1} { __sv }
1857                 { \_numodel_emit_stock_valve:nn {##1} {##2} }
1858         }
1859     % Stock-as-rate phantom-valve nodes (source stock has no matching
1860     % outflow term, so the inflow gets a cloud at the open end)
1861     \prop_map_inline:Nn \l__numodel_stock_phantom_valve_prop
1862         {
1863             \tl_if_in:nnF {##1} { __sv }
1864                 { \_numodel_emit_stock_phantom_valve:nn {##1} {##2} }
1865         }
1866     \typeout{BUILD:~step~6~flow~arrows}
1867     % Flow arrows (one segment each; drawn underneath the valves)
1868     \tl_gput_right:NV \g__numodel_graphic_tl \l__numodel_flows_tl
1869     \typeout{BUILD:~step~6a~valve~nodes}
1870     % Valve nodes (white-filled, drawn ON TOP of the flow arrows)
1871     \tl_gput_right:NV \g__numodel_graphic_tl \l__numodel_valves_tl
1872     \typeout{BUILD:~step~6b+7~causal~arrows~(Lua)}
1873     % A2: causal arrows via Lua - emit_causals demultiplexes on
1874     % tgt_is_valve and pushes to \g__numodel_graphic_tl or
1875     % \l__numodel_late_causals_tl respectively. Afterwards append
1876     % late-causals to graphic_tl once: that covers both the pushes
1877     % from the flow-builders (step 6/6a) and those from emit_causals.
1878     \_numodel_lua_emit_causals:
1879     \tl_gput_right:NV \g__numodel_graphic_tl \l__numodel_late_causals_tl
1880     \typeout{BUILD:~step~8~done}
1881     \tl_gput_right:Nn \g__numodel_graphic_tl
1882         { \end{tikzpicture} }

```



```

1883 }
1884
1885 % --- Node placement ---
1886 \cs_new_protected:Npn \__numodel_place_node:n #1
1887 {
1888   \bool_set_true:N \l_tmpa_bool
1889   \tl_set:Ne \l__numodel_scratch_tl { \use:c { #1 type } }
1890   \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { system }
1891     { \bool_set_false:N \l_tmpa_bool }
1892   \bool_if:NT \l_tmpa_bool
1893     {
1894       \int_compare:nNnT { \use:c { #1 gridx } } = { -1 }
1895         { \bool_set_false:N \l_tmpa_bool }
1896     }
1897   \bool_if:NT \l_tmpa_bool
1898     {
1899       \tl_set:Ne \l__numodel_tmp_tl { \use:c { #1 text } }
1900       % With forrester|edu, valve vars get a dual emission:
1901       % the natural aux/const node + a phantom valve next to the stock.
1902       \bool_set_false:N \l_tmpb_bool % is this a valve var?
1903       \prop_if_in:NnT \l__numodel_valve_for_prop {#1}
1904         { \bool_set_true:N \l_tmpb_bool }
1905       \prop_if_in:NnT \l__numodel_outvalve_for_prop {#1}
1906         { \bool_set_true:N \l_tmpb_bool }
1907       \prop_if_in:NnT \l__numodel_between_valve_prop {#1}
1908         { \bool_set_true:N \l_tmpb_bool }
1909       \bool_lazy_and:nnTF
1910         { \l__numodel_keep_natural_bool } { \l_tmpb_bool }
1911         { \__numodel_emit_natural_and_phantom:n {#1} }
1912         {
1913           \prop_if_in:NnTF \l__numodel_valve_for_prop {#1}
1914             { \__numodel_emit_valve:n {#1} }
1915             {
1916               \prop_if_in:NnTF \l__numodel_outvalve_for_prop {#1}
1917                 { \__numodel_emit_outvalve:n {#1} }
1918                 {
1919                   \prop_if_in:NnTF \l__numodel_between_valve_prop {#1}
1920                     { \__numodel_emit_between_valve:n {#1} }
1921                     {
1922                       \tl_set:Ne \l__numodel_scratch_tl { \use:c { #1 type } }
1923                       \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { stock }
1924                         { \__numodel_emit_stock:n {#1} }
1925                       \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { aux }
1926                         { \__numodel_emit_aux:n {#1} }
1927                       \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { constant }
1928                         { \__numodel_emit_const:n {#1} }
1929                     }
1930                   }
1931                 }
1932             }
1933         }
1934     }
1935
1936 % --- Helper: emit a coordinate node at the valve position ---
1937 % The flow arrow starts/ends at this coordinate (or at an offset
1938 % relative to it, for the open-end without cloud). The visible
1939 % white-filled valve is added later (valves_tl) at the same
1940 % coordinates so it overlays the arrow.
1941 % #1 = coord id #2 = x #3 = y
1942 \cs_new_protected:Npn \__numodel_emit_valve_coord:nnn #1 #2 #3
1943 {
1944   \tl_gput_right:Ne \g__numodel_graphic_tl
1945     {
1946       \exp_not:N \coordinate ~
1947       (#1) ~ \exp_not:n{at} ~ (#2 , ~ #3) ;
1948     }

```

```

1949 }
1950
1951 % --- Helper: append a valve node to the deferred valves_tl ---
1952 % Renders \node[<valve-style>] (id) at (x,y) {<label>}; later in the
1953 % picture so the white fill overlays the flow arrow underneath.
1954 % #1 = node id #2 = x coord #3 = y coord #4 = label tl name
1955 \cs_new_protected:Npn \__numodel_defer_valve:nnnn #1 #2 #3 #4
1956 {
1957   \tl_put_right:Ne \l__numodel_valves_tl
1958   {
1959     \exp_not:N \node ~ [ \tl_use:N \l__numodel_valve_node_tl ] ~
1960     (#1) ~ \exp_not:n{at} ~ (#2 , ~ #3) ~
1961     {
1962       \bool_if:NTF \l__numodel_valve_label_bool
1963       { \exp_not:N $ \exp_not:V #4 \exp_not:N $ }
1964       { }
1965     } ;
1966   }
1967 }
1968
1969 % --- Forrester/edu-mode emission: natural node + phantom valve ---
1970 % The variable sits as an ordinary aux/const node at its own gridy.
1971 % The phantom valve (id #1__v) sits next to the stock at gridy=0; its
1972 % appearance follows the resolved valve-style (label included only
1973 % when valve-style=edu). A causal arrow from #1 -> #1__v is drawn
1974 % (deferred to late_causals_tl so it lands on top of the white-
1975 % filled valve).
1976 \tl_new:N \l__numodel_phantom_label_tl
1977 \cs_new_protected:Npn \__numodel_emit_natural_and_phantom:n #1
1978 {
1979   % --- 1. Natural node (aux or constant) ---
1980   \tl_set:Ne \l__numodel_scratch_tl { \use:c { #1 type } }
1981   \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { aux }
1982   { \__numodel_emit_aux:n {#1} }
1983   \exp_args:NV \str_if_eq:nnT \l__numodel_scratch_tl { constant }
1984   { \__numodel_emit_const:n {#1} }
1985   % --- 2. Locate the phantom valve x/y position ---
1986   \prop_get:NnNT \l__numodel_vpos_x_prop {#1} \l__numodel_phantom_x_tl
1987   {
1988     \prop_get:NnNF \l__numodel_vpos_y_prop {#1} \l__numodel_scratch_y_tl
1989     { \tl_set:Nn \l__numodel_scratch_y_tl { 0 } }
1990     \tl_set:Ne \l__numodel_phantom_label_tl { \use:c { #1 text } }
1991     % --- 3. Coordinate at the phantom valve position ---
1992     \__numodel_emit_valve_coord:nnn { #1 __vc }
1993     { \tl_use:N \l__numodel_phantom_x_tl }
1994     { \tl_use:N \l__numodel_scratch_y_tl }
1995     % --- 4. Flow arrow + (optional) cloud, one segment ---
1996     \prop_if_in:NnTF \l__numodel_valve_for_prop {#1}
1997     {
1998       \tl_set:Ne \l__numodel_tmp_tl
1999       { \prop_item:Nn \l__numodel_valve_for_prop {#1} }
2000       \__numodel_eff_flowcloud:V \l__numodel_tmp_tl
2001       \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2002       {
2003         \tl_put_right:Ne \l__numodel_flows_tl
2004         {
2005           \exp_not:N \flowarrow [#1 __cl] {#1 __vc}
2006           { \tl_use:N \l__numodel_tmp_tl }
2007         }
2008       }
2009       {
2010         \tl_put_right:Ne \l__numodel_flows_tl
2011         {
2012           \exp_not:N \flowarrow {#1 __vc}
2013           { \tl_use:N \l__numodel_tmp_tl }
2014         }
2015       }
2016     }
2017   }

```

```

2015     }
2016     \_numodel_emit_valve_extras:n {#1}
2017 }
2018 {
2019     \prop_if_in:NnTF \l__numodel_outvalve_for_prop {#1}
2020     {
2021         \tl_set:Ne \l__numodel_tmp_tl
2022         { \prop_item:Nn \l__numodel_outvalve_for_prop {#1} }
2023         \_numodel_eff_flowcloud:V \l__numodel_tmp_tl
2024         \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2025         {
2026             \tl_put_right:Ne \l__numodel_flows_tl
2027             {
2028                 \exp_not:N \flowoutarrow [#1 __cl]
2029                 { \tl_use:N \l__numodel_tmp_tl } {#1 __vc}
2030             }
2031         }
2032         {
2033             \tl_put_right:Ne \l__numodel_flows_tl
2034             {
2035                 \exp_not:N \flowoutarrow
2036                 { \tl_use:N \l__numodel_tmp_tl } {#1 __vc}
2037             }
2038         }
2039         \_numodel_emit_outvalve_extras:n {#1}
2040     }
2041     {
2042         % between (no cloud: both ends are stocks)
2043         \tl_set:Ne \l__numodel_tmp_tl
2044         { \prop_item:Nn \l__numodel_between_valve_prop {#1} }
2045         \tl_set:Ne \l__numodel_scratch_tl
2046         { \prop_item:Nn \l__numodel_between_target_prop {#1} }
2047         \tl_put_right:Ne \l__numodel_flows_tl
2048         {
2049             \exp_not:N \flowbetweenarrow
2050             { \tl_use:N \l__numodel_tmp_tl }
2051             {#1 __vc}
2052             { \tl_use:N \l__numodel_scratch_tl }
2053         }
2054     }
2055 }
2056 % --- 5. Defer the phantom valve drawing (overlays flow) ---
2057 \_numodel_defer_valve:nnnn { #1 __v }
2058 { \tl_use:N \l__numodel_phantom_x_tl }
2059 { \tl_use:N \l__numodel_scratch_y_tl }
2060 \l__numodel_phantom_label_tl
2061 % --- 6. Causal arrow natural -> phantom valve (deferred) ---
2062 \tl_put_right:Ne \l__numodel_late_causals_tl
2063 {
2064     \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2065     (#1) ~ \exp_not:n{to} ~ (#1 __v) ;
2066 }
2067 }
2068 }
2069 \tl_new:N \l__numodel_phantom_x_tl
2070 \cs_generate_variant:Nn \_numodel_eff_flowcloud:n { V }
2071
2072 \cs_new_protected:Npn \_numodel_emit_stock:n #1
2073 {
2074     \tl_gput_right:Ne \g__numodel_graphic_tl
2075     {
2076         \exp_not:N \node ~ [\exp_not:n{stock}] ~
2077         (#1) ~ \exp_not:n{at} ~
2078         ( \use:c{#1 gridx} , ~ \use:c{#1 gridy} ) ~
2079         { \exp_not:N $ \exp_not:V \l__numodel_tmp_tl \exp_not:N $ } ;
2080     }

```

```

2081 }
2082
2083 % Tight-mode helpers: the valve sits at the variable's own gridx/
2084 % gridy. We emit a coordinate node (#1__vc) early so the flow
2085 % arrow has a positioning anchor, and defer the visible valve node
2086 % (#1) to valves_tl so it overlays the arrow with white fill.
2087 \cs_new_protected:Npn \__numodel_emit_tight_valve_setup:n #1
2088 {
2089   \__numodel_emit_valve_coord:nnn { #1 __vc }
2090   { \use:c{#1 gridx} } { \use:c{#1 gridy} }
2091   \__numodel_defer_valve:nnnn {#1}
2092   { \use:c{#1 gridx} } { \use:c{#1 gridy} }
2093   \l__numodel_tmp_tl
2094 }
2095
2096 \cs_new_protected:Npn \__numodel_emit_valve:n #1
2097 {
2098   \__numodel_emit_tight_valve_setup:n {#1}
2099   \tl_set:Nx \l__numodel_scratch_tl
2100     { \prop_item:Nn \l__numodel_valve_for_prop {#1} }
2101   \__numodel_eff_flowcloud:V \l__numodel_scratch_tl
2102   \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2103   {
2104     \tl_put_right:Nx \l__numodel_flows_tl
2105       {
2106         \exp_not:N \flowarrow [#1 __cl] {#1 __vc}
2107         { \tl_use:N \l__numodel_scratch_tl }
2108       }
2109   }
2110   {
2111     \tl_put_right:Nx \l__numodel_flows_tl
2112       {
2113         \exp_not:N \flowarrow {#1 __vc}
2114         { \tl_use:N \l__numodel_scratch_tl }
2115       }
2116   }
2117   \__numodel_emit_valve_extras:n {#1}
2118 }
2119
2120 % Emit one curved inflow branch per shared-inflow extra target
2121 % (\l__numodel_valve_extras_prop[fv]). Each branch starts at the
2122 % valve coordinate and sweeps over the primary stock to the extra
2123 % target's north edge. No-op when the valve is not shared.
2124 \cs_new_protected:Npn \__numodel_emit_valve_extras:n #1
2125 {
2126   \prop_get:NnNT \l__numodel_valve_extras_prop {#1} \l__numodel_scratch_tl
2127   {
2128     \clist_set:NV \l__numodel_scratch_clist \l__numodel_scratch_tl
2129     \clist_map_inline:Nn \l__numodel_scratch_clist
2130     {
2131       \tl_put_right:Nn \l__numodel_flows_tl
2132         { \flowarrowbent {#1 __vc} {##1} }
2133     }
2134   }
2135 }
2136
2137 % Mirror of \__numodel_emit_valve_extras:n for shared outflow valves.
2138 % Each extra source stock gets a curved branch arcing over the
2139 % intermediate stocks into the shared valve's open end.
2140 \cs_new_protected:Npn \__numodel_emit_outvalve_extras:n #1
2141 {
2142   \prop_get:NnNT \l__numodel_outvalve_extras_prop
2143     {#1} \l__numodel_scratch_tl
2144   {
2145     \clist_set:NV \l__numodel_scratch_clist \l__numodel_scratch_tl
2146     \clist_map_inline:Nn \l__numodel_scratch_clist

```

```

2147     {
2148         \tl_put_right:Nn \l__numodel_flows_tl
2149         { \flowoutarrowbent {#1} {#1 __vc} }
2150     }
2151 }
2152 }
2153 \clist_new:N \l__numodel_scratch_clist
2154
2155 \cs_new_protected:Npn \__numodel_emit_outvalve:n #1
2156 {
2157     \__numodel_emit_tight_valve_setup:n {#1}
2158     \tl_set:Ne \l__numodel_scratch_tl
2159     { \prop_item:Nn \l__numodel_outvalve_for_prop {#1} }
2160     \__numodel_eff_flowcloud:V \l__numodel_scratch_tl
2161     \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2162     {
2163         \tl_put_right:Ne \l__numodel_flows_tl
2164         {
2165             \exp_not:N \flowoutarrow [#1 __cl]
2166             { \tl_use:N \l__numodel_scratch_tl } {#1 __vc}
2167         }
2168     }
2169     {
2170         \tl_put_right:Ne \l__numodel_flows_tl
2171         {
2172             \exp_not:N \flowoutarrow
2173             { \tl_use:N \l__numodel_scratch_tl } {#1 __vc}
2174         }
2175     }
2176     \__numodel_emit_outvalve_extras:n {#1}
2177 }
2178
2179 \cs_new_protected:Npn \__numodel_emit_between_valve:n #1
2180 {
2181     \__numodel_emit_tight_valve_setup:n {#1}
2182     \tl_set:Ne \l__numodel_scratch_tl
2183     { \prop_item:Nn \l__numodel_between_valve_prop {#1} }
2184     \tl_set:Ne \l__numodel_tmp_tl
2185     { \prop_item:Nn \l__numodel_between_target_prop {#1} }
2186     \tl_put_right:Ne \l__numodel_flows_tl
2187     {
2188         \exp_not:N \flowbetweenarrow
2189         { \tl_use:N \l__numodel_scratch_tl }
2190         {#1 __vc}
2191         { \tl_use:N \l__numodel_tmp_tl }
2192     }
2193 }
2194
2195 \cs_new_protected:Npn \__numodel_emit_aux:n #1
2196 {
2197     \tl_gput_right:Ne \g__numodel_graphic_tl
2198     {
2199         \exp_not:N \node ~ [\exp_not:n{aux}] ~
2200         (#1) ~ \exp_not:n{at} ~
2201         ( \use:c{#1 gridx} , ~ \use:c{#1 gridy} ) ~
2202         { \exp_not:N $ \exp_not:V \l__numodel_tmp_tl \exp_not:N $ } ;
2203     }
2204 }
2205
2206 \cs_new_protected:Npn \__numodel_emit_const:n #1
2207 {
2208     \tl_gput_right:Ne \g__numodel_graphic_tl
2209     {
2210         \exp_not:N \constnode
2211         {#1}
2212         { \use:c{#1 gridx} , ~ \use:c{#1 gridy} }

```

```

2213     { \exp_not:N $ \exp_not:V \l__numodel_tmp_tl \exp_not:N $ }
2214   }
2215 }
2216
2217 % --- Stock-as-flow valve emission ---
2218 \cs_new_protected:Npn \__numodel_emit_stock_valve:nn #1 #2
2219 {
2220   % #1 = target stock (e.g. modH), #2 = source stock (e.g. modV)
2221   % Pull gridx from the saved property
2222   \prop_get:NnNTF \l__numodel_stock_valve_prop { #1 __svgx }
2223   \l__numodel_scratch_tl
2224   {
2225     % Look up the wrap-shifted y-slot; default to 0 (the row that
2226     % the auto layout used before any gridmaxx wrap was applied).
2227     \prop_get:NnNF \l__numodel_stock_valve_prop { #1 __svgy }
2228     \l__numodel_scratch_y_tl
2229     { \tl_set:Nn \l__numodel_scratch_y_tl { 0 } }
2230     % Coordinate at the valve position (used by the flow arrow)
2231     \__numodel_emit_valve_coord:nnn { #1 __svc }
2232     { \tl_use:N \l__numodel_scratch_tl }
2233     { \tl_use:N \l__numodel_scratch_y_tl }
2234     % Flow arrow from source-stock through valve into target-stock
2235     % (no cloud: both ends are stocks). The arrow runs in one
2236     % continuous segment; the visible valve will be drawn ON TOP
2237     % later via valves_tl.
2238     \tl_put_right:Ne \l__numodel_flows_tl
2239     {
2240       \exp_not:N \flowbetweenarrow {#2} {#1 __svc} {#1}
2241     }
2242     % Defer the visible valve node (white fill overlays arrow)
2243     \tl_set:Ne \l__numodel_tmp_tl { \use:c { #2 text } }
2244     \__numodel_defer_valve:nnnn { #1 __sv }
2245     { \tl_use:N \l__numodel_scratch_tl }
2246     { \tl_use:N \l__numodel_scratch_y_tl }
2247     \l__numodel_tmp_tl
2248     % Causal arrow source-stock -> valve. Bend it (curved) only
2249     % when the source stock and the valve sit on the same row -
2250     % then the straight causal would coincide with the flow pipe
2251     % and disappear behind it. When they are on different rows
2252     % (e.g. after a gridmaxx wrap) the causal is vertical-ish and
2253     % stays visible without a bend. Deferred so it lands on top
2254     % of the white-filled valve.
2255     \str_if_eq:eeTF { \tl_use:N \l__numodel_scratch_y_tl }
2256       { \use:c { #2 gridy } }
2257     {
2258       \tl_put_right:Ne \l__numodel_late_causals_tl
2259       {
2260         \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2261         (#2) ~ \exp_not:n{to} ~ [\exp_not:n{bend-left=30}] ~ (#1__sv) ;
2262       }
2263     }
2264     {
2265       \tl_put_right:Ne \l__numodel_late_causals_tl
2266       {
2267         \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2268         (#2) ~ \exp_not:n{to} ~ (#1__sv) ;
2269       }
2270     }
2271   }
2272   { } % no gridx found: skip
2273 }
2274
2275 % --- Stock phantom-valve emission (source stock as rate factor) ---
2276 % Source stock acts as a rate factor for the target stock without a
2277 % matching outflow term; render with a cloud-fed inflow valve next to
2278 % the target stock and link the source stock via a causal arrow.

```

```

2279 \cs_new_protected:Npn \__numodel_emit_stock_phantom_valve:nn #1 #2
2280 {
2281   % #1 = target stock (e.g. ballY), #2 = source stock (e.g. ballV)
2282   \prop_get:NnNTF \l__numodel_stock_phantom_valve_prop { #1 __svgx }
2283   \l__numodel_scratch_tl
2284   {
2285     % Look up the wrap-shifted y-slot; default to 0.
2286     \prop_get:NnNF \l__numodel_stock_phantom_valve_prop { #1 __svgy }
2287     \l__numodel_scratch_y_tl
2288     { \tl_set:Nn \l__numodel_scratch_y_tl { 0 } }
2289     % Coordinate at the valve position
2290     \__numodel_emit_valve_coord:nnn { #1 __svc }
2291     { \tl_use:N \l__numodel_scratch_tl }
2292     { \tl_use:N \l__numodel_scratch_y_tl }
2293     % Flow arrow: cloud (open end) -> valve -> target stock
2294     \__numodel_eff_flowcloud:n {#1}
2295     \str_if_eq:VnTF \l__numodel_flowcloud_eff_tl { true }
2296     {
2297       \tl_put_right:Ne \l__numodel_flows_tl
2298       {
2299         \exp_not:N \flowarrow [#1 __svcl] {#1 __svc} {#1}
2300       }
2301     }
2302     {
2303       \tl_put_right:Ne \l__numodel_flows_tl
2304       {
2305         \exp_not:N \flowarrow {#1 __svc} {#1}
2306       }
2307     }
2308     % Defer the visible valve node, labelled with the source stock
2309     \tl_set:Ne \l__numodel_tmp_tl { \use:c { #2 text } }
2310     \__numodel_defer_valve:nnnn { #1 __sv }
2311     { \tl_use:N \l__numodel_scratch_tl }
2312     { \tl_use:N \l__numodel_scratch_y_tl }
2313     \l__numodel_tmp_tl
2314     % Causal arrow source-stock -> valve. Bend (curved) only when
2315     % source and valve sit on the same row - see the equivalent
2316     % comment in \__numodel_emit_stock_valve:nn for the rationale.
2317     \str_if_eq:eeTF { \tl_use:N \l__numodel_scratch_y_tl }
2318     { \use:c { #2 gridy } }
2319     {
2320       \tl_put_right:Ne \l__numodel_late_causals_tl
2321       {
2322         \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2323         (#2) ~ \exp_not:n{to} ~ [\exp_not:n{bend~left=30}] ~ (#1__sv) ;
2324       }
2325     }
2326     {
2327       \tl_put_right:Ne \l__numodel_late_causals_tl
2328       {
2329         \exp_not:N \draw ~ [\exp_not:n{causal}] ~
2330         (#2) ~ \exp_not:n{to} ~ (#1__sv) ;
2331       }
2332     }
2333   }
2334   { } % no gridx found: skip
2335 }
2336
2337 % -----
2338 % Pillar A - Lua-side layout writeback. A single directlua call
2339 % that runs compute_layout (build_deps, classify_flows,
2340 % populate_occupied, auto_layout, causals) and writes the resulting
2341 % state back to the TeX-side props (\<var>gridx/gridy and the
2342 % flow-props) that the downstream emitters (place_node,
2343 % flow-builders, emit_natural_and_phantom, emit_stock_valve) read.
2344 % Lua is the single source of truth for the decision logic; TeX

```

```

2345 % only renders.
2346 % -----
2347 \cs_new_protected:Npn \__numodel_lua_layout_writeback:
2348 {
2349     % 1. Clear the props that Lua refills, so repeated \graphicmodel
2350     %     calls don't produce duplicate entries.
2351     \prop_clear:N \l__numodel_valve_for_prop
2352     \prop_clear:N \l__numodel_valve_extras_prop
2353     \prop_clear:N \l__numodel_outvalve_for_prop
2354     \prop_clear:N \l__numodel_outvalve_extras_prop
2355     \prop_clear:N \l__numodel_between_valve_prop
2356     \prop_clear:N \l__numodel_between_target_prop
2357     \prop_clear:N \l__numodel_stock_valve_prop
2358     \prop_clear:N \l__numodel_stock_phantom_valve_prop
2359     \prop_clear:N \l__numodel_vpos_x_prop
2360     \prop_clear:N \l__numodel_vpos_y_prop
2361     % 2. Run the Lua pipeline.
2362     \directlua{ numodel.compute_layout(
2363         "\g_numodel_current_prefix_tl",
2364         "\g__numodel_diagram_style_tl",
2365         \int_use:N \g_numodel_gridmaxx_int) }
2366     % 3. Write gridx/gridy and all flow-props back.
2367     %     tex_writeback emits a list of expl3 statements via tex.print.
2368     \directlua{ numodel.tex_writeback(
2369         "\g_numodel_current_prefix_tl") }
2370 }
2371
2372 \cs_new_protected:Npn \__numodel_lua_emit_causals:
2373 {
2374     \directlua{ numodel.emit_causals(
2375         "\g_numodel_current_prefix_tl") }
2376 }
2377
2378 \NewDocumentCommand{\graphicmodel}{ O{} }{%
2379     \tl_clear:N \l__numodel_cmd_prefix_tl
2380     \tl_clear:N \l__numodel_cmd_diagstyle_tl
2381     \tl_clear:N \l__numodel_cmd_flowarrow_tl
2382     \tl_clear:N \l__numodel_cmd_valve_tl
2383     \tl_clear:N \l__numodel_cmd_flowcloud_tl
2384     \keys_set:nn { numodel / cmd } {#1}
2385     \__numodel_resolve_eff_prefix:
2386     % Temporary overrides: save the global state, apply the per-call
2387     % key values before build, restore afterwards. This way
2388     % \graphicmodel[diagram-style=forrester] flips one render without
2389     % touching the global \numodelsetup state.
2390     \tl_set_eq:NN \l__numodel_saved_diagstyle_tl \g__numodel_diagram_style_tl
2391     \tl_set_eq:NN \l__numodel_saved_flowarrow_tl \g__numodel_flowarrow_style_tl
2392     \tl_set_eq:NN \l__numodel_saved_valve_tl \g__numodel_valve_style_tl
2393     \tl_set_eq:NN \l__numodel_saved_flowcloud_tl \g__numodel_flowcloud_tl
2394     \tl_if_empty:NF \l__numodel_cmd_diagstyle_tl
2395     { \tl_gset_eq:NN \g__numodel_diagram_style_tl \l__numodel_cmd_diagstyle_tl }
2396     \tl_if_empty:NF \l__numodel_cmd_flowarrow_tl
2397     { \tl_gset_eq:NN \g__numodel_flowarrow_style_tl \l__numodel_cmd_flowarrow_tl }
2398     \tl_if_empty:NF \l__numodel_cmd_valve_tl
2399     { \tl_gset_eq:NN \g__numodel_valve_style_tl \l__numodel_cmd_valve_tl }
2400     \tl_if_empty:NF \l__numodel_cmd_flowcloud_tl
2401     { \tl_gset_eq:NN \g__numodel_flowcloud_tl \l__numodel_cmd_flowcloud_tl }
2402     % Resolve the effective values for this render and propagate them
2403     % to \nmflowbody / \nmflowtip used by the flow macros.
2404     \__numodel_resolve_flowarrow:
2405     \__numodel_resolve_valve:
2406     \__numodel_resolve_flowcloud:
2407     \__numodel_apply_flowarrow_style:
2408     \__numodel_apply_valve_style:
2409     \__numodel_with_prefix:Nn \l__numodel_eff_prefix_tl
2410     {

```



```

2411 \__numodel_build_graphic:
2412 \tl_use:N \g__numodel_graphic_tl
2413 }%
2414 \tl_gset_eq:NN \g__numodel_diagram_style_tl \l__numodel_saved_diagstyle_tl
2415 \tl_gset_eq:NN \g__numodel_flowarrow_style_tl \l__numodel_saved_flowarrow_tl
2416 \tl_gset_eq:NN \g__numodel_valve_style_tl \l__numodel_saved_valve_tl
2417 \tl_gset_eq:NN \g__numodel_flowcloud_tl \l__numodel_saved_flowcloud_tl
2418 }
2419 \tl_new:N \l__numodel_saved_diagstyle_tl
2420 \tl_new:N \l__numodel_saved_flowarrow_tl
2421 \tl_new:N \l__numodel_saved_valve_tl
2422 \tl_new:N \l__numodel_saved_flowcloud_tl
2423
2424 % =====
2425 % Plot machinery
2426 % =====
2427 % \diagrammodel calls the pgfplots/calcplothdims infrastructure
2428 % through internal wrappers \__numodel_calc_plot_dims: and
2429 % \__numodel_draw_plot:n. Those wrappers forward to \calcplothdims
2430 % and \drawplot from the sibling package numodel-plot.
2431 %
2432 % The wrappers are not \cs_new_eq:NN because numodel-plot can load
2433 % after this package; at load time \calcplothdims and \drawplot may
2434 % not yet exist. With a wrapper they are looked up at expansion
2435 % time.
2436
2437 \cs_new:Npn \__numodel_calc_plot_dims: { \calcplothdims }
2438 \cs_new:Npn \__numodel_draw_plot:n #1 { \drawplot {#1} }
2439
2440 % =====
2441 % \diagrammodel -- convenience command for model graphs
2442 % =====
2443 % Usage: \diagrammodel{xvar}{yvar}[extra drawplot code]{label}
2444 % \diagrammodel{xvar}{yvar1,yvar2,...,yvarn}[extra]{label}
2445 %
2446 % Builds a complete figure with model points. Sets axis dimensions
2447 % from min/max, axis labels from text/unit, and emits caption and
2448 % label automatically.
2449 %
2450 % When the second argument is a comma-separated list, every y-variable
2451 % whose unit matches the first one is drawn into the same diagram with
2452 % the y-axis scaled to the joint min/max range. Variables with a
2453 % mismatching unit are dropped with a warning. Every series is drawn
2454 % as discrete model points (no connecting lines); the seven colours
2455 % come from Okabe & Ito's colour-blind safe palette (yellow omitted)
2456 % and are ordered so consecutive series differ in luminance, which
2457 % keeps them distinguishable on a greyscale printout. The legend
2458 % lists each kept variable's display text.
2459
2460 \tl_new:N \l__numodel_xname_tl
2461 \tl_new:N \l__numodel_yname_tl
2462 \tl_new:N \l__numodel_yfull_tl
2463 \tl_new:N \l__numodel_yunit_first_tl
2464 \tl_new:N \l__numodel_yqty_tl
2465 \tl_new:N \l__numodel_plotbody_tl
2466 \clist_new:N \l__numodel_yshort_clist
2467 \int_new:N \l__numodel_yidx_int
2468
2469 % Colour-blind safe palette (Okabe & Ito; yellow #F0E442 omitted for
2470 % poor white-background contrast). Ordered so consecutive series
2471 % differ in luminance as well as hue, keeping them distinguishable
2472 % on a greyscale printout.
2473 \definecolor{numodelseriesa}{HTML}{000000} % black (L ~ 0.00)
2474 \definecolor{numodelseriesb}{HTML}{E69F00} % orange (L ~ 0.69)
2475 \definecolor{numodelseriesc}{HTML}{0072B2} % dark blue (L ~ 0.36)
2476 \definecolor{numodelseriesd}{HTML}{56B4E9} % sky blue (L ~ 0.74)

```

```

2477 \definecolor{numodelseriese}{HTML}{D55E00} % vermillion      (L ~ 0.50)
2478 \definecolor{numodelseriesf}{HTML}{CC79A7} % reddish purple  (L ~ 0.62)
2479 \definecolor{numodelseriesg}{HTML}{009E73} % bluish green    (L ~ 0.52)
2480
2481 \clist_new:N \g__numodel_series_colors_clist
2482 \clist_gset:Nn \g__numodel_series_colors_clist
2483 {
2484     numodelseriesa , numodelseriesb , numodelseriesc , numodelseriesd ,
2485     numodelseriese , numodelseriesf , numodelseriesg
2486 }
2487
2488 % Return the cycling colour name for series #1 (1-based), modulo 7.
2489 \cs_new:Npn \__numodel_series_color:n #1
2490 {
2491     \clist_item:Nn \g__numodel_series_colors_clist
2492     { 1 + \int_mod:nn { #1 - 1 } { 7 } }
2493 }
2494
2495 \NewDocumentCommand{\diagrammodel}{ O{} m m O{} m }
2496 {
2497     \tl_clear:N \l__numodel_cmd_prefix_tl
2498     \keys_set:nn { numodel / cmd } {#1}
2499     \__numodel_resolve_eff_prefix:
2500     \tl_set:Ne \l__numodel_xname_tl { \l__numodel_eff_prefix_tl #2 }
2501     % --- Parse comma-separated y-variable list -----
2502     % Keep only variables whose unitraw matches the first one's; warn
2503     % about the rest.
2504     \clist_clear:N \l__numodel_yshort_clist
2505     \tl_clear:N \l__numodel_yunit_first_tl
2506     \int_zero:N \l__numodel_yidx_int
2507     \clist_map_inline:nn {#3}
2508     {
2509         \int_incr:N \l__numodel_yidx_int
2510         \tl_set:Ne \l__numodel_yfull_tl { \l__numodel_eff_prefix_tl ##1 }
2511         \int_compare:nNnTF { \l__numodel_yidx_int } = { 1 }
2512         {
2513             \tl_set:Ne \l__numodel_yunit_first_tl
2514             { \use:c { \l__numodel_yfull_tl unitraw } }
2515             \clist_put_right:Nn \l__numodel_yshort_clist {##1}
2516         }
2517         {
2518             \tl_set:Ne \l_tmpa_tl
2519             { \use:c { \l__numodel_yfull_tl unitraw } }
2520             \str_if_eq:VVTf \l_tmpa_tl \l__numodel_yunit_first_tl
2521             { \clist_put_right:Nn \l__numodel_yshort_clist {##1} }
2522             {
2523                 \msg_warning:nnee { numodel } { unit-mismatch }
2524                 { \tl_use:N \l__numodel_yfull_tl }
2525                 { \tl_use:N \l__numodel_yunit_first_tl }
2526             }
2527         }
2528     }
2529     % Canonical y-name = first kept variable (legacy single-series
2530     % accessor used by the caption when only one series is plotted).
2531     \clist_get:NN \l__numodel_yshort_clist \l_tmpa_tl
2532     \tl_set:Ne \l__numodel_yname_tl
2533     { \l__numodel_eff_prefix_tl \l_tmpa_tl }
2534     % --- Axis ranges -----
2535     \def\xmin{\use:c{\l__numodel_xname_tl min}}
2536     \def\xmax{\use:c{\l__numodel_xname_tl max}}
2537     \def\xlabelqty{\use:c{\l__numodel_xname_tl text}}
2538     \def\xlabelunit{\use:c{\l__numodel_xname_tl unitraw}}
2539     % Reduce over the kept y-variables. Use \edef + \fpeval so the
2540     % per-variable min/max accessors are expanded to numeric literals
2541     % before the fp parser sees them.
2542     \def\ymin{inf}

```

```

2543 \def\ymax{-inf}
2544 \clist_map_inline:Nn \l__numodel_yshort_clist
2545 {
2546   \tl_set:N \l__numodel_yfull_tl { \l__numodel_eff_prefix_tl ##1 }
2547   \edef\ymin{\fpeval{min(\ymin,\use:c{\l__numodel_yfull_tl min})}}
2548   \edef\ymax{\fpeval{max(\ymax,\use:c{\l__numodel_yfull_tl max})}}
2549 }
2550 % --- y-axis label: comma-joined display texts, common unit -----
2551 \tl_clear:N \l__numodel_yqty_tl
2552 \int_zero:N \l__numodel_yidx_int
2553 \clist_map_inline:Nn \l__numodel_yshort_clist
2554 {
2555   \int_incr:N \l__numodel_yidx_int
2556   \int_compare:nNnF { \l__numodel_yidx_int } = { 1 }
2557   { \tl_put_right:Nn \l__numodel_yqty_tl { ,~ } }
2558   \tl_put_right:Ne \l__numodel_yqty_tl
2559   { \use:c { \l__numodel_eff_prefix_tl ##1 text } }
2560 }
2561 \edef\ylabelqty{\tl_use:N \l__numodel_yqty_tl}
2562 \edef\ylabelunit{\tl_use:N \l__numodel_yunit_first_tl}
2563 \begin{figure}[H]
2564 \centering
2565 \group_begin:
2566 \__numodel_apply_dsep:
2567 % --- Build the plot body -----
2568 \int_compare:nNnTF
2569 { \clist_count:N \l__numodel_yshort_clist } = { 1 }
2570 {
2571   % Single-series: preserve the legacy mark-only rendering.
2572   \edef\dm@coords
2573   { \mcoordsp{\l__numodel_eff_prefix_tl}{#2}{\tl_use:N \l_tmpa_tl} }
2574   \__numodel_draw_plot:n
2575   {
2576     \addplot[only-marks,~mark=*,~mark-size=0.5pt]~coordinates~{\dm@coords};
2577     \addlegendentry{model~point}
2578     #4
2579   }
2580 }
2581 {
2582   % Multi-series: one mark-only \addplot per kept variable, with
2583   % cycling colour-blind safe colour, and a text legend. The
2584   % legend's mark is enlarged so the colour stays visible there.
2585   \tl_clear:N \l__numodel_plotbody_tl
2586   \int_zero:N \l__numodel_yidx_int
2587   \clist_map_inline:Nn \l__numodel_yshort_clist
2588   {
2589     \int_incr:N \l__numodel_yidx_int
2590     \tl_put_right:Ne \l__numodel_plotbody_tl
2591     {
2592       \exp_not:N \addplot
2593       [ only~marks ,~mark=*,~mark-size=0.5pt ,
2594         color = \__numodel_series_color:n
2595           { \int_use:N \l__numodel_yidx_int } ,
2596         legend-image~post~style = { mark-size=2pt } ]
2597       coordinates {
2598         \mcoordsp { \l__numodel_eff_prefix_tl } { #2 } { ##1 }
2599       } ;
2600       \exp_not:N \addlegendentry
2601       { $ \use:c { \l__numodel_eff_prefix_tl ##1 text } $ }
2602     }
2603   }
2604   \tl_put_right:Nn \l__numodel_plotbody_tl {#4}
2605   \exp_args:NV \__numodel_draw_plot:n \l__numodel_plotbody_tl
2606 }
2607 \group_end:
2608 \caption{$\tl_use:N \l__numodel_yqty_tl$(\use:c{\l__numodel_xname_tl text})$\text{-diagram}}

```

```

2609 % If the project-specific worksheet system is loaded, pick the
2610 % '-ws' label inside a worksheet; otherwise always the bare label.
2611 \cs_if_exist:NTF \ifinworksheet
2612 { \ifinworksheet{\label{fig:#5-ws}}{\label{fig:#5}} }
2613 { \label{fig:#5} }
2614 \end{figure}
2615 }
2616
2617 \ExplSyntaxOff
2618
2619 % =====
2620 % GRAPHIC MODEL - STYLES AND MACROS
2621 % =====
2622 %
2623 % Forrester-diagram conventions:
2624 %
2625 % Stock [stock] Rectangle v, s, T, x
2626 % Valve / inflow [valve] Circle on a thick a, dv, ds
2627 % flow pipe
2628 % Auxiliary [aux] Plain circle F_res, F_air
2629 % Constant \constnode Circle + dashes m, k, g
2630 %
2631 % Use \dgridx and \dgridy as the grid spacings for consistent positioning.
2632 % =====
2633
2634 % == Grid spacing -- defaults set via \numodelsetup {graphscalex, graphscaley} ==
2635
2636 % == Styles ==
2637 \tikzset{
2638 % Scale coordinates to the grid spacing
2639 gridscale/.style={x=\dgridx cm, y=\dgridy cm},
2640 % Stock (state variable): rectangle
2641 stock/.style={
2642 rectangle, draw, thick,
2643 minimum width=3 em,
2644 minimum height=2 em,
2645 font=\small
2646 },
2647 % Auxiliary (intermediate variable): plain circle
2648 aux/.style={
2649 circle, draw, thick,
2650 inner sep=2pt,
2651 minimum size=2 em,
2652 font=\scriptsize
2653 },
2654 % Constant: circle (dashes drawn by the \constnode macro)
2655 const/.style={
2656 circle, draw, thick,
2657 inner sep=2pt,
2658 minimum size=2 em,
2659 font=\scriptsize
2660 },
2661 % Valve (legacy alias): circle on the flow pipe. Kept for
2662 % backwards compatibility with user code that calls \flowarrow
2663 % manually. The package's emit helpers now pick one of the
2664 % style variants below based on \numodelsetup{valve-style=...}.
2665 valve/.style={
2666 circle, draw, thick, fill=white,
2667 minimum size=2 em,
2668 font=\small
2669 },
2670 % Valve variant: Forrester valve symbol -- two filled triangles
2671 % (white inside) whose tips meet at the centre. The symmetry
2672 % axis stands perpendicular to the (horizontal) flow. The
2673 % bounding box is tall and narrow so the apex angle at the
2674 % centre is roughly 60 degrees (sharper than the 90 degrees of

```

```

2675 % a square box), and the horizontal "ribs" along the top and
2676 % bottom edges close the two triangles.
2677 valve-forrester/.style={
2678   shape=rectangle,
2679   draw=none, fill=none,
2680   inner sep=0pt, outer sep=0pt,
2681   %minimum width=\dgridx*0.18 cm,
2682   minimum width=1 em,
2683   %minimum height=\dgridy*0.30 cm,
2684   minimum height=2 em,
2685   path picture={%
2686     \draw[thick, fill=white]
2687       (path picture bounding box.north west) --
2688       (path picture bounding box.north east) --
2689       (path picture bounding box.center) --
2690       cycle;
2691     \draw[thick, fill=white]
2692       (path picture bounding box.south west) --
2693       (path picture bounding box.south east) --
2694       (path picture bounding box.center) --
2695       cycle;
2696   }
2697 },
2698 % Valve variant: empty circle (no label).
2699 valve-circle/.style={
2700   circle, draw, thick, fill=white,
2701   minimum size=2 em,
2702   font=\small
2703 },
2704 % Valve variant: circle with the variable's display label inside.
2705 valve-edu/.style={
2706   circle, draw, thick, fill=white,
2707   minimum size=2 em,
2708   font=\small
2709 },
2710 % Flow pipe variants (body without arrow tip, used for the
2711 % segment from open-end / cloud to the valve):
2712 flowpipe/.style={line width=3pt}, % legacy alias
2713 flowpipe-filled/.style={line width=3pt},
2714 flowpipe-hollow/.style={
2715   line width=1pt,
2716   double=white, double distance=3pt
2717 },
2718 % Flow pipe variants WITH arrow tip (used for the segment from
2719 % valve to stock, or the open-end side of an outflow). Both
2720 % variants use the Stealth arrow tip. The hollow form
2721 % uses the double-line magic length 0pt-3-0 so that the arrow
2722 % tip merges cleanly with the gap between the two strokes.
2723 flowpipe-filled-tip/.style={line width=3pt, arrows={-Stealth[inset=0pt, angle=30:1em]}},
2724 flowpipe-hollow-tip/.style={
2725   line width=1pt,
2726   double=white, double distance=3pt,
2727   arrows={-Stealth[inset=0pt, angle=30:1em]}
2728 },
2729 % Flow pipe variants with the Cloud arrow tip on the open end:
2730 % inflow keeps the Stealth arrow head at the stock side.
2731 flowpipe-filled-cloudin/.style={
2732   line width=3pt,
2733   arrows={Cloud[fill=white, line width=0.8pt]-Stealth[inset=0pt, angle=30:1em]}
2734 },
2735 flowpipe-hollow-cloudin/.style={
2736   line width=1pt,
2737   double=white, double distance=3pt,
2738   arrows={Cloud[fill=white, line width=0.8pt]-Stealth[inset=0pt, angle=30:1em]}
2739 },
2740 % Outflow with cloud at the open end: first a Stealth tip at the

```

```

2741 % line end, then the cloud behind it (in arrows.meta spec, list
2742 % multiple tips separated by a dot so the shaft stops at the
2743 % Stealth tip and does not extend into the cloud; order = from
2744 % line outwards, hence Stealth.Cloud).
2745 flowpipe-filled-cloudout/.style={
2746   line width=3pt,
2747   arrows={-Stealth[inset=0pt, angle=30:1em].Cloud[fill=white, line width=0.8pt]}}
2748 },
2749 flowpipe-hollow-cloudout/.style={
2750   line width=1pt,
2751   double=white, double distance=3pt,
2752   arrows={-Stealth[inset=0pt, angle=30:1em].Cloud[fill=white, line width=0.8pt]}}
2753 },
2754 % Causal arrow (thin arrow for dependencies)
2755 causal/.style={thick, arrows={-Stealth[length=0.5em]}}
2756 }
2757
2758 % Default values for the body / tip macros so a manual call to
2759 % \flowarrow (outside \graphicmodel) keeps working. The
2760 % \graphicmodel pipeline overwrites these per render.
2761 \providecommand{\nmflowbody}{flowpipe-filled}
2762 \providecommand{\nmflowtip}{flowpipe-filled-tip}
2763 \providecommand{\nmflowtipcloudin}{flowpipe-filled-cloudin}
2764 \providecommand{\nmflowtipcloudout}{flowpipe-filled-cloudout}
2765
2766 % === Macro: constant node with dashes ===
2767 % Usage: \constnode{name}{(x,y)}{$label$}
2768 \newcommand{\constnode}[3]{%
2769   \node[const] (#1) at (#2) {#3};%
2770   \draw[thick] ([xshift=-3mm]#1.west) -- ([xshift=3mm]#1.east);%
2771   \draw[thick] (#1.east) -- ([xshift=3mm]#1.east);%
2772 }
2773
2774 % === Macro: flow pipe with valve (inflow) ===
2775 % Draws ONE continuous flow arrow ending at the stock. The starting
2776 % position is the same regardless of cloud presence; passing any
2777 % non-blank optional argument switches to the cloud-tipped variant
2778 % so the arrow's tail becomes a white-filled cloud (drawn as a real
2779 % PGF arrow tip, not a separate node - moves with the line). The
2780 % valve overlays the middle of the arrow as a separate white-filled
2781 % node drawn later by the build pipeline.
2782 % Style keys: \nmflowtip / \nmflowtipcloudin.
2783 % Usage: \flowarrow[<any>]{<valve-coord>}{<stock-node>}
2784 \NewDocumentCommand{\flowarrow}{ O{} m m }{%
2785   \IfBlankTF{#1}{%
2786     \draw[\nmflowtip] ([xshift=-3em]#2) -- ([xshift=0.5mm]#3.west);%
2787   }{%
2788     \draw[\nmflowtipcloudin] ([xshift=-3em]#2) -- ([xshift=0.5mm]#3.west);%
2789   }%
2790 }
2791
2792 % === Macro: flow pipe with valve (outflow) ===
2793 % Mirror of \flowarrow. The arrow always ends at the same offset
2794 % past the valve coordinate; passing a non-blank optional argument
2795 % switches to the cloud-tipped variant (cloud at the open end).
2796 % Usage: \flowoutarrow[<any>]{<stock-node>}{<valve-coord>}
2797 \NewDocumentCommand{\flowoutarrow}{ O{} m m }{%
2798   \IfBlankTF{#1}{%
2799     \draw[\nmflowtip] (#2.east) -- ([xshift=4em]#3);%
2800   }{%
2801     \draw[\nmflowtipcloudout] (#2.east) -- ([xshift=5em]#3);%
2802   }%
2803 }
2804
2805 % === Macro: flow pipe between two stocks ===
2806 % One continuous arrow from source stock through valve coord into

```

```

2807 % target stock. No cloud option (no open end exists when both
2808 % sides are stocks).
2809 % Usage: \flowbetweenarrow{<source-stock>}{<valve-coord>}{<target-stock>}
2810 \newcommand{\flowbetweenarrow}[3]{%
2811 \draw[nmflowtip] (#1.east) -- ([xshift=0.5mm]#3.west);%
2812 }
2813
2814 % === Macro: curved inflow branch for a shared valve ===
2815 % Used when one inflow valve feeds more than one stock. The primary
2816 % target is drawn with the standard straight \flowarrow; every
2817 % secondary target is drawn with this macro, which arcs UNDER the
2818 % intervening stock(s). Stocks sit on the bottom row of the
2819 % Forrester diagram, so the area below is empty -- routing the
2820 % branch below avoids collisions with the aux/const rows and the
2821 % causal arrows that live above. Both endpoints are passed
2822 % unanchored so TikZ picks the border-intersection that best
2823 % matches the curve's tangent at each end.
2824 % Usage: \flowarrowbent{<valve-coord>}{<target-stock>}
2825 \newcommand{\flowarrowbent}[2]{%
2826 \draw[nmflowtip] (#1) to[bend right=30] (#2);%
2827 }
2828
2829 % === Macro: curved outflow branch for a shared valve ===
2830 % Mirror of \flowarrowbent. Used when one outflow valve drains more
2831 % than one stock. Routes the branch BELOW the intervening stocks
2832 % (same rationale as the inflow variant: nothing else lives below
2833 % the stocks row). Endpoints unanchored so TikZ chooses the
2834 % cleanest border intersection.
2835 % Usage: \flowoutarrowbent{<source-stock>}{<valve-coord>}
2836 \newcommand{\flowoutarrowbent}[2]{%
2837 \draw[nmflowtip] (#1) to[bend right=30] (#2);%
2838 }
2839 \endpackage

```

9.1 Translation files

Each `numodel-<LANG>.def` file populates the lookup table consulted by `\__numodel_kw:n`. The package loads exactly the file matching the current syntax key, via `\InputIfFileExists` (and hence `kpse`), so additional languages can be supplied through `TEXMFHOME/tex/latex/numodel/numodel-<LANG>.def` with no package rebuild. The two shipped files follow.

9.1.1 numodel-EN.def – English (XMILE)

XMILE v1.0 OASIS style: ALL-CAPS, case sensitive. Sets the default decimal mark to `point`.

```

2840 (*EN)
2841 \ProvidesExplFile{numodel-EN.def}{2026/05/23}{0.5.0}
2842 {numodel-keyword-table~---English~(XMILE)}
2843 \cs_new:Npn \__numodel_kw_EN_if: { IF~ }
2844 \cs_new:Npn \__numodel_kw_EN_then: { ~THEN~ }
2845 \cs_new:Npn \__numodel_kw_EN_then_nl: { ~THEN }
2846 \cs_new:Npn \__numodel_kw_EN_else: { ~ELSE~ }
2847 \cs_new:Npn \__numodel_kw_EN_else_nl: { ELSE }
2848 \cs_new:Npn \__numodel_kw_EN_endif: { ~ENDIF }
2849 \cs_new:Npn \__numodel_kw_EN_endif_nl: { ENDIF }
2850 \cs_new:Npn \__numodel_kw_EN_and: { ~AND~ }
2851 \cs_new:Npn \__numodel_kw_EN_or: { ~OR~ }
2852 \cs_new:Npn \__numodel_kw_EN_not: { NOT~ }
2853 \cs_new:Npn \__numodel_kw_EN_sign: { SIGN }
2854 \cs_new:Npn \__numodel_kw_EN_abs: { ABS }
2855 \cs_new:Npn \__numodel_kw_EN_sqrt: { SQRT }
2856 \cs_new:Npn \__numodel_kw_EN_exp: { EXP }
2857 \cs_new:Npn \__numodel_kw_EN_ln: { LN }
2858 \cs_new:Npn \__numodel_kw_EN_sin: { SIN }
2859 \cs_new:Npn \__numodel_kw_EN_cos: { COS }
2860 \cs_new:Npn \__numodel_kw_EN_tan: { TAN }
2861 \cs_new:Npn \__numodel_kw_EN_asin: { ARCSIN }

```

```

2862 \cs_new:Npn \__numodel_kw_EN_acos:      { ARCCOS }
2863 \cs_new:Npn \__numodel_kw_EN_stop:      { ~STOP~ }
2864 \cs_new:Npn \__numodel_kw_EN_th_model:   { model }
2865 \cs_new:Npn \__numodel_kw_EN_th_initvals: { initial~values }
2866 \cs_new:Npn \__numodel_kw_EN_contfoot:   { Continued~on~next~page }
2867 \cs_new:Npn \__numodel_kw_EN_conthead:   { (Continued) }
2868 \cs_new:Npn \__numodel_kw_EN_dsep_default: { point }
2869 </EN>

```

9.1.2 numodel-NL.def – Dutch (CoachTaal)

CoachTaal convention. Sets the default decimal mark to comma.

```

2870 <*NL>
2871 \ProvidesExplFile{numodel-NL.def}{2026/05/23}{0.5.0}
2872 {numodel~keyword~table~---Dutch~(CoachTaal)}
2873 \cs_new:Npn \__numodel_kw_NL_if:          { Als~ }
2874 \cs_new:Npn \__numodel_kw_NL_then:        { ~Dan~ }
2875 \cs_new:Npn \__numodel_kw_NL_then_nl:     { ~Dan }
2876 \cs_new:Npn \__numodel_kw_NL_else:        { ~Anders~ }
2877 \cs_new:Npn \__numodel_kw_NL_else_nl:     { Anders }
2878 \cs_new:Npn \__numodel_kw_NL_endif:       { ~EindAls }
2879 \cs_new:Npn \__numodel_kw_NL_endif_nl:    { EindAls }
2880 \cs_new:Npn \__numodel_kw_NL_and:         { ~EN~ }
2881 \cs_new:Npn \__numodel_kw_NL_or:          { ~OF~ }
2882 \cs_new:Npn \__numodel_kw_NL_not:         { NIET~ }
2883 \cs_new:Npn \__numodel_kw_NL_sign:        { Teken }
2884 \cs_new:Npn \__numodel_kw_NL_abs:         { Abs }
2885 \cs_new:Npn \__numodel_kw_NL_sqrt:       { Sqrt }
2886 \cs_new:Npn \__numodel_kw_NL_exp:        { Exp }
2887 \cs_new:Npn \__numodel_kw_NL_ln:         { Ln }
2888 \cs_new:Npn \__numodel_kw_NL_sin:        { Sin }
2889 \cs_new:Npn \__numodel_kw_NL_cos:        { Cos }
2890 \cs_new:Npn \__numodel_kw_NL_tan:        { Tan }
2891 \cs_new:Npn \__numodel_kw_NL_asin:       { Arcsin }
2892 \cs_new:Npn \__numodel_kw_NL_acos:       { Arccos }
2893 \cs_new:Npn \__numodel_kw_NL_stop:       { ~Stop~ }
2894 \cs_new:Npn \__numodel_kw_NL_th_model:    { model }
2895 \cs_new:Npn \__numodel_kw_NL_th_initvals: { startwaarden }
2896 \cs_new:Npn \__numodel_kw_NL_contfoot:    { Wordt~vervolgd~op~volgende~pagina }
2897 \cs_new:Npn \__numodel_kw_NL_conthead:    { (Vervolg) }
2898 \cs_new:Npn \__numodel_kw_NL_dsep_default: { comma }
2899 </NL>

```